

A Laboratory Manual for

Microcontroller and

Microprocessor

(BE05041011)

B.E. Semester 5

(Robotics and Automation)

L. D. COLLEGE OF ENGINEERING, AHMEDABAD



Directorate of Technical Education,

Gandhinagar, Gujarat

L. D. COLLEGE OF ENGINEERING, AHMEDABAD

Certificate

This is to certify that Mr./Ms. _____
_____ Enrollment No. _____ of B.E. Semester 5, Branch-
Robotics and automation of this Institute (GTU Code: 028) has satisfactorily
completed the Practical / Tutorial work for the subject **Microcontroller and
Microprocessor (BE05041011)** for the academic year 2026-27.

Place: _____

Date: _____

Name and Sign of Faculty member

Head of the Department

Preface

Main motto of any laboratory/practical/field work is for enhancing required skills as well as creating ability amongst students to solve real time problem by developing relevant competencies in psychomotor domain. By keeping in view, GTU has designed competency focused outcome-based curriculum for engineering degree programs where sufficient weightage is given to practical work. It shows importance of enhancement of skills amongst the students and it pays attention to utilize every second of time allotted for practical amongst students, instructors and faculty members to achieve relevant outcomes by performing the experiments rather than having merely study type experiments. It is must for effective implementation of competency focused outcome-based curriculum that every practical is keenly designed to serve as a tool to develop and enhance relevant competency required by the various industry among every student. These psychomotor skills are very difficult to develop through traditional chalk and board content delivery method in the classroom. Accordingly, this lab manual is designed to focus on the industry defined relevant outcomes, rather than old practice of conducting practical to prove concept and theory.

By using this lab manual students can go through the relevant theory and procedure in advance before the actual performance which creates an interest and students can have basic idea prior to performance. This in turn enhances pre-determined outcomes amongst students. Each experiment in this manual begins with competency, industry relevant skills, course outcomes as well as practical outcomes (objectives). The students will also achieve safety and necessary precautions to be taken while performing practical.

This manual also provides guidelines to faculty members to facilitate student centric lab activities through each experiment by arranging and managing necessary resources in order that the students follow the procedures with required safety and necessary precautions to achieve the outcomes. It also gives an idea that how students will be assessed by providing rubrics.

Microprocessor is a field of study that involves the designed to provide hands-on experience with the Intel 8085 microprocessor, a foundational component in the field of digital electronics and computer architecture. The 8085 microprocessor is widely used in various applications, and a thorough understanding of its architecture and programming is essential for anyone aspiring to work in the field of embedded systems, microprocessor programming, or computer engineering.

Utmost care has been taken while preparing this lab manual however always there is chances of improvement. Therefore, we welcome constructive suggestions for improvement and removal of errors if any.

Practical – Course Outcome matrix

Sr. No.	Objective(s) of Experiment	CO 1	CO 2	CO 3	CO 4
1.	Introduction to Microprocessor and identify the different peripheral devices	√			
2.	Assembly language programming using data transfer instructions.	√	√		
3.	Assembly language programming using arithmetic and logical instructions.	√	√		
4.	Assembly language programming using branching and looping instructions.		√		
5.	Perform data transfer programming in 8051	√	√	√	
6.	To study and perform arithmetic instruction programming in 8051			√	
7.	To study and perform Boolean & logical instructions programming.			√	
8.	C program for blinking LED using 8051.		√	√	
9.	Assembly language programming for arithmetic and logical operation.			√	√
10.	C programming to Read push pull button switch and display its status on LED				√
11.	Interfacing buzzer with AVR board using simullIDE.				√
12.	Interfacing 7 segment display with AVR board using simullIDE				√
	Suggested Project / Case study				

Industry Relevant Skills

The following industry relevant competency are expected to be developed in the student by undertaking the practical work of this laboratory.

1. Assembly Language Programming:

- *Skill Developed:* Proficiency in writing and understanding assembly language programs for the 8085 microprocessor.
- *Industry Relevance:* Many embedded systems and microprocessor-based applications require low-level programming skills. Understanding assembly language is crucial for programming microprocessors and optimizing code for resource-constrained environments.

2. Embedded System Design:

- *Skill Developed:* Ability to design and implement simple embedded systems using the 8085 microprocessor.
- *Industry Relevance:* Embedded systems are pervasive in various industries, from consumer electronics to automotive systems. Knowing how to design and interface with microprocessors is fundamental to working in the field of embedded systems.

3. System Architecture Understanding:

- *Skill Developed:* In-depth understanding of the internal architecture of the 8085 microprocessors, including registers, buses, and control signals.
- *Industry Relevance:* Understanding system architecture is crucial for designing efficient and optimized systems. This skill is valuable in roles related to computer architecture, hardware design, and system-level integration.

4. Problem Solving and Debugging:

- *Skill Developed:* Proficiency in troubleshooting and debugging assembly language programs for the 8085 microprocessors.
- *Industry Relevance:* Debugging skills are essential in any technical field. Learning to identify and fix issues in microprocessor programs enhances problem-solving abilities, a critical skill in software development and hardware design.

5. Peripheral Interfacing:

- *Skill Developed:* Experience in interfacing the 8085 microprocessors with external devices and peripherals.
- *Industry Relevance:* Interfacing microprocessors with peripherals is a common task in industries such as robotics, automation, and IoT. Developing skills in this area is beneficial for roles involving hardware-software integration and system-level design.

Guidelines for Faculty members:

1. Teacher should provide the guideline with demonstration of practical to the students with all features.
2. Teacher shall explain basic concepts/theory related to the experiment to the students before starting of each practical

Microcontroller and Microprocessor(BE05041011)

3. Involve all the students in performance of each experiment.
4. Teacher is expected to share the skills and competencies to be developed in the students and ensure that the respective skills and competencies are developed in the students after the completion of the experimentation.
5. Teachers should give opportunity to students for hands-on experience after the demonstration.
6. Teacher may provide additional knowledge and skills to the students even though not covered in the manual but are expected from the students by concerned industry.
7. Give practical assignment and assess the performance of students based on task assigned to check whether it is as per the instructions or not.
8. Teacher is expected to refer complete curriculum of the course and follow the guidelines for implementation.

Instructions for Students:

1. Students are expected to carefully listen to all the theory classes delivered by the faculty members and understand the COs, content of the course, teaching and examination scheme, skill set to be developed etc.
2. Students shall organize the work in the group and make record of all observations.
3. Students shall develop maintenance skill as expected by industries.
4. Student shall attempt to develop related hand-on skills and build confidence.
5. Student shall develop the habits of evolving more ideas, innovations, skills etc. apart from those included in scope of manual.
6. Student shall refer technical magazines and data books.
7. Student should develop a habit of submitting the experimentation work as per the schedule and s/he should be well prepared for the same.

Common Safety Instructions

1. Students are expected to handle computers and trainer kits with care and caution.

(Progressive Assessment Sheet)

Microcontroller and Microprocessor(BE05041011)

Sr. No.	Objective(s) of Experiment	Page No.	Date of performance	Date of submission	Assessment Marks	Sign. Of Teacher with date	Remarks
1	Introduction to Microprocessor and identify the different peripheral devices and operating system on it. Write and execute demo programs.						
2	Assembly language programming using data transfer instructions.						
3	Assembly language programming using arithmetic and logical instructions.						
4	Assembly language programming using branching and looping instructions.						
5	To study and perform data transfer programming in 8051						
6	To study and perform arithmetic instruction programming in 8051						
7	To study and perform Boolean & logical instructions programming.						
8	Write a program for blinking LED using 8051.						
9	Introduction to AVR and assembly language programming for arithmetic and logical operation.						
10	Read push pull button switch and display its status on LED						
11	Interfacing buzzer with AVR board using simullIDE.						
12	Interfacing 7 segment display with AVR board using simullIDE						
Total							

Experiment No: 1

Introduction to Microprocessor and identify the different peripheral devices and operating system on it. Write and execute demo programs.

Date:

Competency and Practical Skills: Competency will be developed in understanding microprocessors, identifying peripheral devices, and comprehending the associated operating system. Additionally, practical skills in writing and executing programs will be acquired.

Relevant CO:CO1

Objectives: To develop a comprehensive understanding of microprocessors, identify peripheral devices, and execute demo programs to demonstrate practical proficiency.

Equipment/Instruments/Software: <https://www.sim8085.com>. However, other software or microprocessor kits or educational boards may also be used.

Theory:

8085 is pronounced as “eighty-eighty-five” microprocessor. It is an 8-bit microprocessor designed by Intel in 1977 using NMOS technology.

It has the following configuration –

- I. 8-bit data bus
- II. 16-bit address bus, which can address up to 64KB
- III. A 16-bit program counter
- IV. A 16-bit stack pointer
- V. Six 8-bit registers arranged in pairs: BC, DE, HL

Requires: +5V supply to operate at 3.2 MHz single phase clock

Use: It is used in washing machines, microwave ovens, mobile phones, etc.

8085 consists of the following functional units –

The Microprocessor is a programmable semiconductor device. It is made by LSI/VLSI technology which includes on chip ALU, Register array and Control circuit.

- Register Array.
- Bus Organization.
- Timing and Control Unit.
- External Interrupt.
- Serial I/O Control.
- Arithmetic and Logical Unit.
- Accumulator
- Temp. Register.
- Flag Register.
- Instructions Register.

Register Array:

It is a group of 8-bit register. Register B, C, D, E, H, L are general purpose register means user can access all these registers to store the data for manipulation. All are also used as a 16-bit register pair i.e., BC, DE, HL pair for same purpose. User cannot access register W&Z. Also, it is having two 16-bit register SP&PC. SP is used to point the stack memory. It will store data of subroutine program. PC is used to point the program memory.

Bus Organization:

Bus is group of wires which is used to transfer data, address or control system from one place to other place. It will mainly divide into two part Internal Bus & External Bus. Internal is used to transfer the data internally. External bus is of three types Address Bus, Data Bus, Control Bus.

1. **Address Bus:** It is a group of 16-bit which is used to identify the address of memory or I/O device. It is unidirectional means data will transfer from the Microprocessor to peripheral device.
2. **Data Bus:** It is a group of 8-bit which is transfer the data between memory or I/O device. It is a bidirectional means data will transfer in both directions.
3. **Control Bus:** It is used to transfer the control signal i.e., MEMR, MEMWR, IOR, IOW etc.

Timing and Control Unit:

It will generate various timing signal like clock input, clock output, control signal like RD, WR, Ready, HOLD, HLDA, ALE, reset in, Reset out, status signal like S0, S1, IO/M etc.

Clock signal used to interface clock pulse with device which will decide the speed of device. Control signal used to perform various operation like reading from memory, writing into memory same way I/O read & write.

Other signal like Ready, Hold is generated by external device. Reset used to reset the device. Status signal used to indicate various operation like latch, memory read, memory write etc.

External Interrupt:

It is having interrupt pins like RST 5.5, RST 6.5, RST 7.5, TRAP, INTR which is either edge or level sensitive each having particular vector location except INTR which is hardware interrupt.

Whenever any interrupt occurs microprocessor stop the operation and goes to that location of interrupt and perform the particular operation. At the end of subroutine, it will again continue with the previous operation.

Serial I/O Control:

It is used to transfer the data into serial from between microprocessor and peripheral device SID used to accept data serially inside the microprocessor, SOD used to transfer the data outside of microprocessor.

Arithmetic and Logic Unit (ALU):

It performs various 8-bit arithmetical operations like addition, subtraction, increment, and decrement. Logical operation like and, or, ex-or, compliment, compare etc. All these operations use accumulator as one of its operands and the result will be placed into accumulator.

Accumulator:

It is 8-bit register to perform various arithmetical & logical operation within it and result will be store inside it.

Temporary Register:

It will store the data temporarily during arithmetic and logic operations.

Flag Register:

It is having total five flags. Flag is one kind of flip-flop which is set or reset according to the result of arithmetic and logical operations. Flags are as follows CY, AC, P, Z and S, according to flag register microprocessor will take different decision.

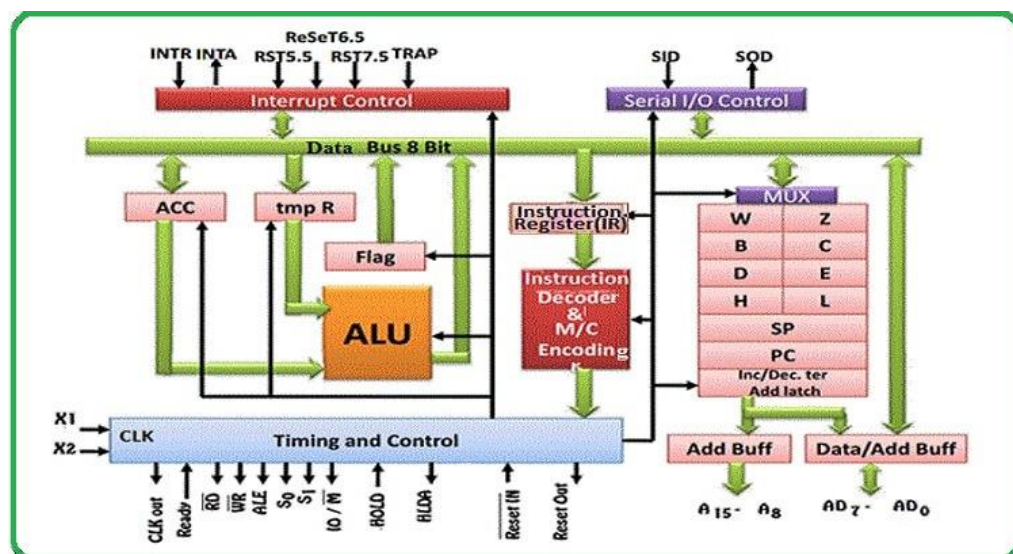
1. CY Flag- It is set whenever result is more than 8-bit.
2. AC Flag- It is set when the carry will be generated from the addition of bit D3 position of the result.
3. P Flag- It is set when result has even no. of ones.
4. Z Flag- It is set when the result is zero.
5. S Flag- It is set when D7 bit of accumulator is 1.

Instruction Register:

Whatever the instruction is fetched by the processor from code memory it will be placed in Instruction register.

Instruction Decoder:

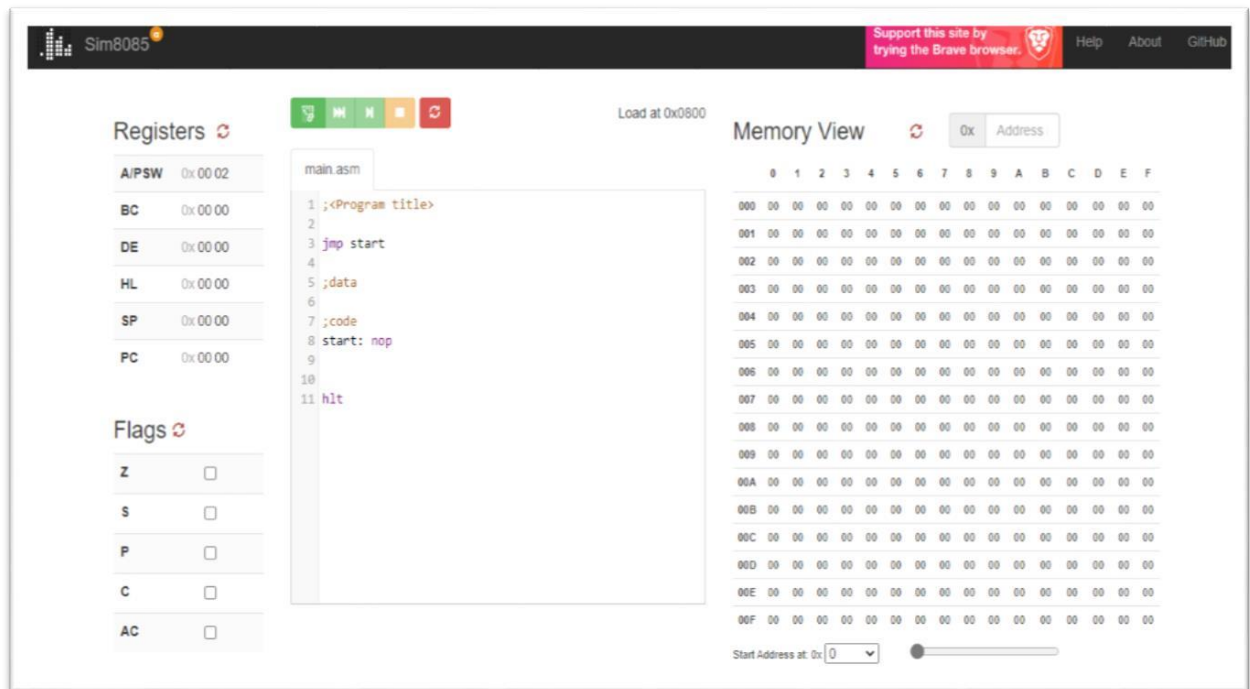
It will decide the instruction and accordingly generate the various timing and control signals.



The Simulator being used for performing Practicals of the subject, “Microprocessor and Interfacing” is “Sim8085”. Sim8085 is an online development environment for writing Intel 8085 microprocessor code. It can assemble, debug 8085 assembly code and simulate the 8085 microprocessors.

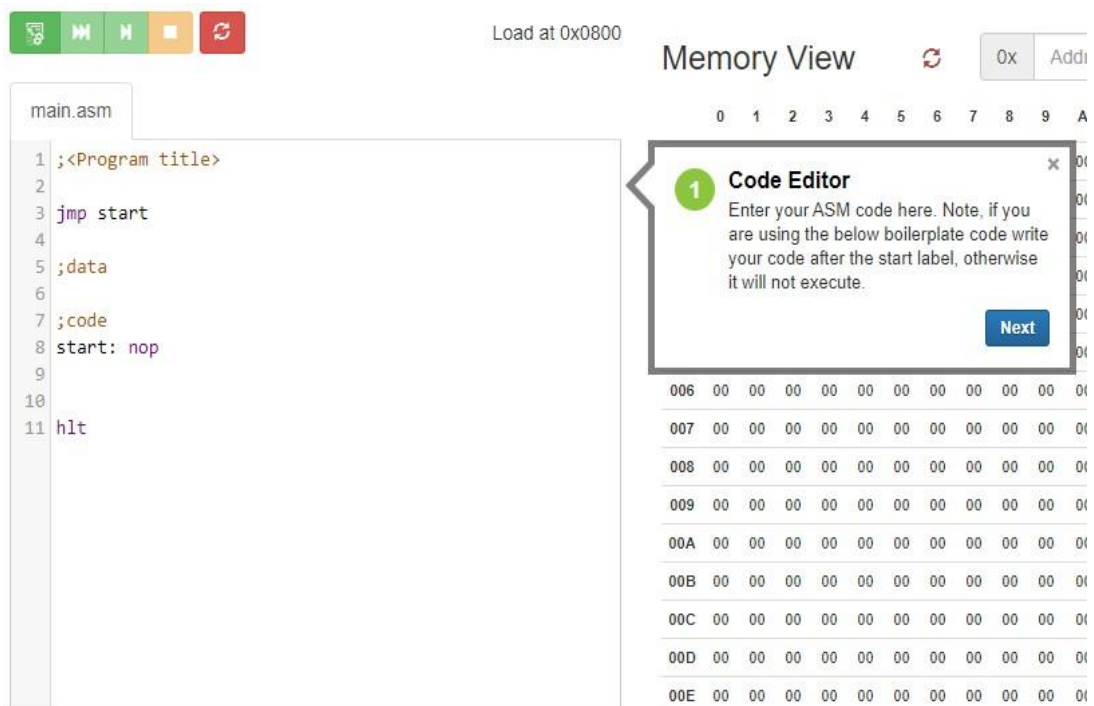
Microcontroller and Microprocessor(BE05041011)

The simulator has two parts, an assembler, which translates 8085 assembly code to machine code and 8085 microprocessor simulator which executes machine code. This web application is a graphical interface for the simulator.



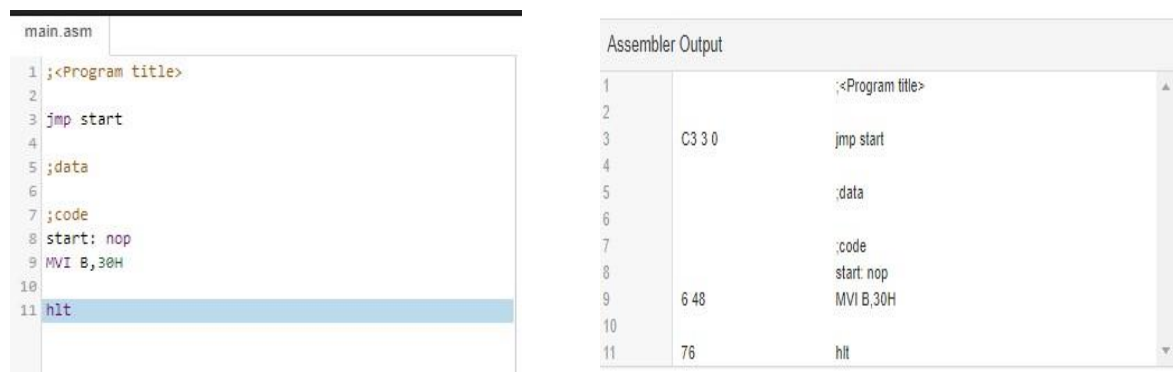
Components of the Microprocessor Simulator:

- 1. Program Code Window:** The Program Code Window is where the Program for the 8085 Simulator is written. A code comprises of “instructions”, which are assembly language keywords which initiate an operation. Various instructions initiate an operation, like data transfer, or arithmetic calculations. More Details about the Instruction set are discussed in Point 4.



2. Compile Buttons: The Compile buttons above the Code Window are used to compile the program. The Codes are first assembled, then compiled. The information of the assembled codes can be seen in the Assembler Output window present at the bottom of the code window.

- The First Green Button assembles and loads the program to the memory.
- The Second and third buttons are for complete execution and executing one instruction of the program respectively.
- The fourth button stops the execution of the code.
- The last button resets the entire compiler and memory.



3. Registers Section: The register section shows the values of registers and allows editing of values by double clicking on the values.



4. Flag Section: The flags section can be used to view the value of the flags as they change and also set or unset them manually.



5. **Memory:** The memory view shows all the 65536 (0xFFFF) memory cells available to 8085. Location/address of a memory can be viewed either by manually moving the scroll bar at the bottom to the desired location or directly searching it at the search bar at the top. Data can be directly added into the memory also.

Memory View 

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
001	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
002	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
003	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
004	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
005	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
006	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
007	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
008	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
009	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at: 0x 

Working of the 8085 Microprocessor Simulator:

The 8085 microprocessor is an 8-bit microprocessor designed by Intel in 1977 using NMOS technology.

It has the following configuration –

- 8-bit data bus
- 16-bit address bus, which can store up to 64KB
- A 16-bit program counter
- A 16-bit stack pointer
- Six 8-bit registers arranged in pairs: BC, DE, HL

On writing the code in the program code window, and the assembler code is converted to Machine code by the assembler, and compiled by the simulator/compiler. While compiling the code, errors in the instructions are notified to the user. After the code is compiled, the user can now execute each instruction one by one or entirely.

After executing each instruction, the desired output, that is, data stored in memory or in registers can be seen.

Example Program:

For the given program shown below, Data Transfer by loading data from memory to Accumulator or Register A, and into registers B, C, and D. After assembling and compiling it, we obtain the values in the Registers.

```

main.asm
1 ;<Program title>
2
3 jmp start
4
5 ;data
6
7 ;code
8 start: nop
9 LDA 0002
10 MVI B,30H
11 MVI C,40H
12 MVI D,50H
13 hlt
    
```

Assembler Output		
1		; <Program title>
2		
3	C3 3 0	jmp start
4		
5		;data
6		
7		;code
8		start: nop
9	3A 2 0	LDA 0002
10	6 48	MVI B,30H
11	E 64	MVI C,40H
12	16 80	MVI D,50H
13	76	hlt

Registers

A/PSW	0x 33 02
BC	0x 30 40
DE	0x 50 00
HL	0x 00 00
SP	0x FF FF
PC	0x 08 0D

Memory View

0x	Address
	0 1 2 3 4 5 6 7 8 9 A B C D E F
000	00 00 33 00 00 00 00 00 00 00 00 00 00 00 00 00
001	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
002	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
003	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Safety and necessary Precautions:

1. Do not start the computer or kit without instructions

Conclusion:

Quiz:

1. What is the various register available in 8085?
2. What are the various flags used in 8085?
3. What is stack pointer?
4. What is program counter?
5. Define interrupt.
6. What is the hardware & software interrupts?
7. What is the clock frequency for 8085?
8. What is ALE? Explain the function of ALE in 8085?

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

1. Knowledge Acquisition: Assess the depth of understanding in microprocessors, peripheral devices, and operating systems.
2. Program Execution: Evaluate the successful execution of demo programs as a practical application of the acquired knowledge.
3. Peripheral Device Identification: Measure the ability to correctly identify and describe different peripheral devices associated with the microprocessor.
4. Operating System Familiarity: Assess the level of comprehension regarding the microprocessor's operating system and its role in the experiment.
5. Overall Proficiency: Determine the overall competency and practical skills demonstrated in the experiment, showcasing a holistic grasp of the topic.

Experiment No: 2

Assembly language programming using data transfer instructions.

Date:

Competency and Practical Skills: Competency in assembly language programming with data transfer instructions will be developed, and practical skills for applying such instructions will be gained.

Relevant CO: CO1, CO2

Objectives: To acquire proficiency in assembly language programming by utilizing data transfer instructions.

Equipment/Instruments/Software: <https://www.sim8085.com>. However, other software or microprocessor kits or educational boards may also be used.

Theory:

We use Microprocessor to solve various types of arithmetic and logical operations. For which we have to write programs. We can write program in assembly language and then it is converted into equivalent machine codes. These machine codes are entered into available microprocessor kit. The process of converting the assembly language program into machine level language (i.e., 1 & 0) is called assembling. If we do this process manually then it is called manual assembling and if we do this using software then it is called assembler.

Data Transfer instructions performs the primary function of copying data from a register or I/O or memory which is called the source, to another register or I/O memory, which is called the destination.

Data transfer instructions can be used to copy Data :

- From register to register.
- Load an 8 bit number in register.
- Between memory and register
- Between I/O and accumulator
- Load 16-bit number in register.

According to different operations of Microprocessor, we can classify the Instructions of Microprocessor as follow.

- DATA TRANSFER OPERATION
- ARITHMETIC OPERATION
- LOGIC OPERATION
- BRANCHING OPERATION
- MACHINE CONTROL OPERATION

DATA TRANSFER OPERATION:

Using data transfer operation Instruction we can transfer the data among registers, between register & memory, between register & I/O, between memory & I/O etc. After transferring data from source to destination, the content of source remains same but the content of destination will be changed. We can use any register, memory or input device as a source and transfer the data to the destination. We can use any register, memory or output device as a destination. Data transfer operation will not affect any flag register.

We transfer the data between various devices using following Instructions.

MOV Rd, Rs [Transfer the data from source register (i.e., A, B, C, D, E, H, L) to any other register same as above.]

- Size; single byte
- Addressing: register
- Machine Cycle : 1 (F)
- T-state: 4

MVI R, 8-bit [Transfer the 8-bit immediate data to any register (i.e., A, B, C, D, E, H, L)]

- Size: two bytes
- Addressing: immediate
- Machine Cycle: 2 (F, MR)
- T-state: 7

MVI M, 8-bit [Transfer the 8-bit immediate data to memory]

- Size: 2 bytes
- Addressing : immediate
- Machine Cycle: 3 (F, MR, MW)
- T-state: 10

LDA Addr [Transfer the data from memory location to accumulator]

- Size: 3 bytes
- Addressing: direct
- Machine Cycle: 4 (F, MR, MR, MR)
- T-state: 13

LDAX Rp [Load a from memory address contain in register pair BC/DE]

Microcontroller and Microprocesso(BE05041011)

- Size: one byte
- Addressing: indirect
- Machine Cycle: 2 (F, MR)
- T-state: 7

STA Addr [Transfer data from accumulator to memory location]

- Size: 3 bytes
- Addressing: direct
- Machine Cycle: 4 (F, MR, MR, MW)
- T-state: 13

STAX Rp [Transfer the data from Accumulator to memory location contain in register pair BC/DE]

- Size: 1 byte
- Addressing: indirect
- Machine Cycle: 2 (F, MW)
- T-state: 7

LXI Rp, 16 bits [Load 16- bit in register pair]

- Size; 3 bytes
- Addressing: immediate
- Machine Cycle: (F, MR, MR)
- T-state: 10

LHLD Address [Load HL Direct]

- Size: 3 bytes
- Addressing: direct
- Machine Cycle: 5 (F, MR, MR ,MR, MR)
- T-STATE: 16

SHLD Address [Store HL direct]

- Size: 3 bytes
- Addressing: direct
- Machine Cycle: 5 (F, MR, MR, MW, MW)
- T-STATE: 16

XTHL [Exchange stack with HL]

- Size: 1 byte

Microcontroller and Microprocesso(BE05041011)

- Addressing: direct
- Machine Cycle: 5 (F, MR, MR, MW, MW)
- T-state: 16

XCHG [Exchange HL with DE]

- Size; 1 byte
- Addressing: register
- Machine Cycle: 1 (F)
- T-state: 4

OUT 8-bit port address[Transfer the data from Accumulator to 8- bit port address (output-device)]

- Size; 2 bytes
- Addressing: register
- Machine Cycle; 3 (F, MR, LOW)
- T-STAE: 10

IN 8-bit port address [Transfer the data from to 8- bit port (input device) address to Accumulator]

- Size: 2 bytes
- Addressing: register
- Machine Cycle: 3 (F, MR, IOR)
- T-state: 10

PUSH Rp [Copy the content of register pair on to stack]

- Size: 1 byte
- Addressing: register
- Machine Cycle: 3 (F, MW, MW)
- T-state: 12

POP Rp [Copy the content of stack to register pair]

- Size: 1 byte]
- Addressing: register
- Machine Cycle: 3 (F, MR, MR)
- T-state: 10

Instructions related to data transfer:

The following notations are used in the description of the instructions.

R = 8 bit register (A,B,C,D,E,H,L)

Microcontroller and Microprocesso(BE05041011)

M = memory register

RS = Register source

RD = Register Destination

RP = Register Pair

Microcontroller and Microprocesso(BE05041011)

Sr. No	Instruction	
	OPCODE	OPERAND
1.	MOV	RD,RS
2.	MOV	M,R
3.	MOV	R,M
4.	MVI	R,8-BITS
5.	MVI	M,8-BITS
6.	LXI	RP,16-BITS
7.	LDA	16-BITS
8.	STA	16-BITS
9.	LDAX	Rp
10.	STAX	Rp
11.	LHLD	16-BITS
12.	SHLD	16-BITS
13.	PCHL	
14.	SPHL	
15.	XCHG	
16.	XTHL	
17.	IN	8 BITS
18.	OUT	8 BITS

Assembly language programs:

1. Load the number 64H in register C and 91H in register A. (Immediate addressing mode.)
(MVI)

MVI C,64H

MVI A,91H

HLT

The screenshot displays an 8085 assembly simulator interface. On the left, the 'Registers' panel shows A/PSW at 0x9102, BC at 0x0064, DE at 0x0000, HL at 0x0000, SP at 0xFFFF, and PC at 0x0800. Below it, the 'Flags' panel shows Z, S, P, C, and AC flags all set to 0. The central 'main.asm' editor contains three lines of assembly code: 1. MVI C,64H, 2. MVI A,91H, and 3. HLT. To the right, the 'Memory View' panel shows a memory dump starting at address 0000, with all bytes currently set to 00. At the bottom, the 'Assembler Output' panel shows the assembly process: line 1 with opcode 0E and hex value 64 for MVI C,64H; line 2 with opcode 3E and hex value 91 for MVI A,91H; and line 3 with opcode 76 for HLT.

Microcontroller and Microprocesso(BE05041011)

2. Store the data 14H, 20H and FFH in registers A, B and C respectively. Copy the same into registers D, E and H respectively. (Immediate addressing mode followed by register addressing mode.) (MVI/MOV)

MVI A,14H

MVI B,20H

MVI C,FFH

MOV D,A

MOV E,B

MOV H,C

HLT

Registers

A/PSW	0x14 02
BC	0x20 FF
DE	0x14 20
HL	0xFF 00
SP	0xFF FF
PC	0x08 00

Flags

Z	☐
S	☐
P	☐
C	☐
AC	☐

main.asm

```
1 MVI A,14H
2 MVI B,20H
3 MVI C,FFH
4
5 MOV D,A
6 MOV E,B
7 MOV H,C
8
9 HLT
```

Memory View

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
001	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
002	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
003	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
004	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
005	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
006	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
007	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
008	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
009	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at: 0x0

Assembler Output

1	3E 14	MVI A,14H
2	06 20	MVI B,20H
3	0E FF	MVI C,FFH
4		
5	57	MOV D,A
6	58	MOV E,B
7	61	MOV H,C
8		
9	76	HLT

3. Interchange the contents of registers B and C. (Register addressing mode.) (MVI/MOV)

MVI B,18H

MVI C,35H

MOV E,B

MOV B,C

MOV C,E

HLT

Registers

A/PSW	0x00 02
BC	0x35 18
DE	0x00 18
HL	0x00 00
SP	0xFF FF
PC	0x08 00

Flags

Z	☐
S	☐
P	☐
C	☐
AC	☐

main.asm

```
1 MVI B,18H
2 MVI C,35H
3
4 MOV E,B
5 MOV B,C
6 MOV C,E
7
8 HLT
```

Memory View

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
001	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
002	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
003	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
004	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
005	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
006	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
007	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
008	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
009	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at: 0x0

Assembler Output

1	06 18	MVI B,18H
2	0E 35	MVI C,35H
3		
4	58	MOV E,B
5	41	MOV B,C
6	4B	MOV C,E
7		
8	76	HLT

Microcontroller and Microprocesso(BE05041011)

4. Store some data at memory location C101H using keyboard. Write an ALP to transfer data from C101H to accumulator and copy it into reg. C. (Direct memory address mode.) (LDA)

LDA C101H

MOV C,A

HLT

Registers

A/PSW 0x1802
BC 0x0018
DE 0x0000
HL 0x0000
SP 0xFF FF
PC 0x0800

Flags

Z ☐
S ☐
P ☐
C ☐
AC ☐

main.asm

```
1 LDA C101H  
2 MOV C,A  
3 HLT
```

Memory View

0x C101H

Start Address at 0x BB00

Assembler Output

Line	Address	Instruction
1	3A 01 c1	LDA C101H
2	4F	MOV C,A
3	76	HLT

5. Using keyboard store data 99H at memory location C100H. Write an ALP to take the data to accumulator, copy it into register B and store it at memory location C110H. (Direct memory address mode and register address mode.) (LDA/MOV)

LDA C100H

MOV B,A

STA C110H

HLT

Registers

A/PSW 0x1802
BC 0x0000
DE 0x0000
HL 0x0000
SP 0xFF FF
PC 0x0800

Flags

Z ☐
S ☐
P ☐
C ☐
AC ☐

main.asm

```
1 LDA C100H  
2 MOV B,A  
3 STA C110H  
4 HLT
```

Memory View

0x C100H

Start Address at 0x BB00

Assembler Output

Line	Address	Instruction
1	3A 00 c1	LDA C100H
2	47	MOV B,A
3	32 10 c1	STA C110H
4	76	HLT

Microcontroller and Microprocessor(BE05041011)

6. Memory location C154H and C155H contain some data XY. Write an ALP to transfer the same to register pair BC and DE. (Direct memory address mode.)

LHLD C154H

XCHG

MOV B,D

MOV C,E

HLT

The screenshot shows an 8085 assembly simulator. The **Registers** panel on the left shows: A/PSW: 0x0002, BC: 0x1804, DE: 0x1804, HL: 0x0000, SP: 0xFFFF, PC: 0x0800. The **Flags** panel shows Z, S, P, C, and AC flags are all cleared. The **Memory View** on the right shows memory locations C10 to C1F, with C154H containing 04 and C155H containing 15. The **Assembler Output** at the bottom shows the following instructions and their addresses:

Line	Address	Instruction
1	2A 54 c1	LHLD C154H
2	E8	XCHG
3	42	MOV B,D
4	4B	MOV C,E
5		
6	76	HLT

7. Using keyboard store some data in memory locations C100H, C101H, C102H & C103H. Copy the same into

a) Registers A, B, C & D respectively (using HL as memory pointer). (MOV A,M)

LHLD C100H

MOV A,L

MOV B,H

LHLD C102H

MOV C,L

MOV D,H

HLT

The screenshot shows an 8085 assembly simulator. The **Registers** panel on the left shows: A/PSW: 0x0402, BC: 0x1835, DE: 0x4800, HL: 0x4835, SP: 0xFFFF, PC: 0x0800. The **Flags** panel shows Z, S, P, C, and AC flags are all cleared. The **Memory View** on the right shows memory locations C10 to C1F, with C100H containing 04, C101H containing 15, C102H containing 35, and C103H containing 48. The **Assembler Output** at the bottom shows the following instructions and their addresses:

Line	Address	Instruction
1	2A 00 c1	LHLD C100H
2	7D	MOV A,L
3	44	MOV B,H
4	2A 02 c1	LHLD C102H
5	4D	MOV C,L
6	54	MOV D,H
7		
8	76	HLT

Microcontroller and Microprocesso(BE05041011)

- b) Registers A, B, C & H respectively (using DE as memory pointer). (LDAX) (Indirect memory addressing mode).

MVI D,C1H

MVI E,03H

LDAX D

MOV H,A

DCX D

LDAX D

MOV C,A

DCX D

LDAX D

MOV B,A

DCX D

LDAX D

HLT

Registers

A/PSW 0x0402
BC 0x1835
DE 0xC100
HL 0x4800
SP 0xFF00
PC 0x0800

Flags

Z ☐
S ☐
P ☐
C ☐
AC ☐

main.asm

```
1: MVI D,C1H
2: MVI E,03H
3: LDAX D
4: MOV H,A
5: DCX D
6: LDAX D
7: MOV C,A
8: DCX D
9: LDAX D
10: MOV B,A
11: DCX D
12: LDAX D
13: HLT
```

Assembler Output

Address	Hex	Op-Code	Instruction
16	c1		MVI D,C1H
1E	03		MVI E,03H
3A			LDAX D
67			MOV H,A
1B			DCX D
3A			LDAX D
4F			MOV C,A
1B			DCX D
3A			LDAX D
47			MOV B,A
1B			DCX D
3A			LDAX D
76			HLT

Memory View

0x C100H

Start Address at 0xBB00

8. To replace the data byte by 00H, Stored at memory location whose address is stored at DE register pair. (STAX)

MVI D,C1H

MVI E,00H

STAX D

HLT

Registers

A/PSW 0x0002
BC 0x0000
DE 0xC100
HL 0x0000
SP 0xFF00
PC 0x0800

Flags

Z ☐
S ☐
P ☐
C ☐
AC ☐

main.asm

```
1: MVI D,C1H
2: MVI E,00H
3: STAX D
4: HLT
```

Assembler Output

Address	Hex	Op-Code	Instruction
16	c1		MVI D,C1H
1E	00		MVI E,00H
32			STAX D
76			HLT

Memory View

0x C100H

Start Address at 0xBB00

Conclusion:

Quiz:

1. Write the names of addressing modes.
2. List any three data transfer instruction with immediate addressing modes.
3. Explain the operation of PUSH & POP instruction with suitable program.

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

1. Proficiency in Instruction Utilization: Assess the ability to effectively employ data transfer instructions in assembly language programming.
2. Error-Free Coding: Evaluate the accuracy of the code, with a focus on minimizing errors related to data transfer.
3. Efficient Algorithm Implementation: Measure the efficiency of the algorithm when using data transfer instructions to manipulate data.
4. Logical Flow: Evaluate the logical flow of the program and the sequence of data transfers within the assembly code.
5. Overall Competency: Determine the overall competence in assembly language programming using data transfer instructions, considering both technical proficiency and problem-solving skills.

Experiment No: 3

Assembly language programming using arithmetic and logical instructions.

Date:

Competency and Practical Skills: Competence in assembly language programming with arithmetic and logical instructions will be achieved, along with practical skills for applying these instructions.

Relevant CO: CO1, CO2

Objectives: To develop proficiency in assembly language programming through the utilization of arithmetic and logical instructions.

Equipment/Instruments/Software: <https://www.sim8085.com>. However, other software or microprocessor kits or educational boards may also be used.

Theory:

Arithmetic operations:

Arithmetic Instructions perform the arithmetic operation such as addition, subtraction, increment, and decrement. Using these instructions we can perform various arithmetic operations like addition, subtraction, increment, decrement etc. All arithmetic operations will be performed with one of the operands in accumulator and result will also be placed into Accumulator. Arithmetic operation will affect the flag register, eg. if result of addition is more than 8-bit carry flag will be set, if we perform sign operation and bit D7(MSB) of result is one then sign flag will be set etc. The instruction that we used as follow.

The instructions used for arithmetic operations are as listed below.

ADD R [Add the content of any register with accumulator and store the result into accumulator]

- Size: one byte
- Addressing: Register
- Machine cycle: 1(F)
- T-state: 4

ADD M [Add the content of memory with accumulator and store the result into accumulator]

- Size: one byte
- Addressing: indirect
- Machine cycle: 2(F,MR)
- T-state: 7

ADD 8-bit [Add the 8-bit immediate data with accumulator and store result into accumulator]

- Size: two bytes

Microcontroller and Microprocesso(BE05041011)

- Addressing: immediate
- Machine cycle: 2(F,MR)
- T-state: 7

ADC R [Add the content of any register & carry with accumulator and store the result into accumulator]

- Size: one byte
- Addressing: register
- Machine cycle: 1(F)
- T-state: 4

ADC M [Add the content of memory & carry with accumulator and store the result into accumulator]

- Size: one byte
- Addressing: indirect
- Machine cycle: 2(F,MR)
- T-state: 7

ACI 8-bit [Add the 8-bit immediate data & carry with Accumulator and store the result into accumulator]

- Size: two bytes
- Addressing: Register
- Machine cycle: 2(F,MR)
- T-state: 7

SUB M [Subtract the content of memory from accumulator and store the result into accumulator]

- Size: one byte
- Addressing: Register
- Machine cycle: 1(F)
- T-state: 4

SUB M [Subtract the content of memory from accumulator and store the result into accumulator]

- Size: one byte
- Addressing: indirect
- Machine cycle: 2(F,MR)
- T-state: 7

SUI 8-bit [Subtract the 8-bit immediate data from accumulator and store the result into accumulator]

- Size: two bytes
- Addressing: immediate
- Machine cycle:2(F,MR)
- T-state: 7

SSB R [Subtract the content of any register & Carry from accumulator and store the result into accumulator]

- Size: one byte
- Addressing: Register
- Machine cycle:1(F)
- T-state: 4

SBB M [Subtract the content of any register & carry from accumulator and store the result into accumulator]

- Size: one byte
- Addressing: Register
- Machine cycle:2(F,MR)
- T-state: 7

DAD Rp [Add register pair to HL and store the result into HL]

- Size: one byte
- Addressing: Register
- Machine cycle:3(F,MR,MR)
- T-state: 10

DAA [Decimal adjust after adding (for BCD arithmetic)]

- Size: one byte
- Addressing: implied
- Machine cycle:1(F)
- T-state: 4

INR R [Increment the content of any register from the initial value by 1]

- Size: one byte
- Addressing: Register

Microcontroller and Microprocesso(BE05041011)

- Machine cycle:1(F)
- T-state: 4

INR M [Increment the content of any memory from the initial value by 1]

- Size: one byte
- Addressing: Register
- Machine cycle:3(F,MR,MW)
- T-state: 10

INX Rp [Increment the content of any register pair from the initial value by 1]

- Size: one byte
- Addressing: Register
- Machine cycle:1(F)
- T-state: 6

DCR R [Decrement the content of any register from the initial value by 1]

- Size: one byte
- Addressing: Register
- Machine cycle:1(F)
- T-state: 4

DCR M [Decrement the content of memory from the initial value by 1]

- Size: one byte
- Addressing: Register
- Machine cycle:2(F,MR,MW)
- T-state: 10

DCX Rp [Decrement the content of any register pair from the initial value by 1]

- Size: one byte
- Addressing: Register
- Machine cycle:1(F)
- T-state: 6

Logical operation:-

Using these instructions, we can perform various logical operations like ANDing, ORing, complimenting, comparing etc. All the logical operation will be performed with one of the operands in accumulator and result will also be placed into accumulator. Logical operation will affect flag register.

Microcontroller and Microprocesso(BE05041011)

The instructions for logic operations are listed below.

ANA R [Logical ANDing any Register with accumulator and result will be place into accumulator]

- Size: one byte
- Addressing: Register
- Machine Cycle: 1 (F)
- T-state: 4

ANA M [Logical ANDing memory content with accumulator and result will be place into accumulator]

- Size: one byte
- Addressing: Indirect
- Machine Cycle: 2 (F,MR)
- T-state: 7

ANI 8-bit [Logical ANDing 8-bit data with accumulator and result will be place into accumulator]

- Size: two bytes
- Addressing: Immediate
- Machine Cycle: 2 (F,MR)
- T-state: 7

ORA R [Logical ORing any register with accumulator and result will be place into accumulator]

- Size: one byte
- Addressing: Register
- Machine Cycle: 1 (F)
- T-state: 4

ORA M [Logical ORing memory content with accumulator and result will be place into accumulator]

- Size: one byte
- Addressing: Indirect
- Machine Cycle: 2 (F,MR)
- T-state: 7

ORI 8-bit [Logical ORing 8-bit data with accumulator and result will be place into accumulator]

- Size: Two byte
- Addressing: Immediate
- Machine Cycle: 2 (F,MR)
- T-state: 7

XRA R [Logical Exoring any register with accumulator and result will be place into accumulator]

- Size: one byte
- Addressing: Register
- Machine Cycle: 1 (F)
- T-state: 4

XRA M [Logical Exoring memory content with accumulator and result will be place into accumulator]

- Size: one byte
- Addressing: Indirect
- Machine Cycle: 2 (F,MR)
- T-state: 7

XRI 8-bit [Logical Exoring 8-bit data with accumulator and result will be place into accumulator]

- Size: two bytes
- Addressing: Immediate
- Machine Cycle: 2 (F,MR)
- T-state: 7

CMA [Compliment Accumulator]

- Size: one byte
- Addressing: Implied
- Machine Cycle: 1 (F)
- T-state: 4

CMC [Compliment Carry]

- Size: one byte
- Addressing: implied
- Machine Cycle: 2 (F,MR)
- T-state: 4

CMP R [Compare any register with accumulator. Content of register & accumulator remains same]

- Size: one byte
- Addressing: register
- Machine Cycle: 1 (F)
- T-state: 4

CMP M [Compare content of memory with accumulator. Content of accumulator remain same]

Microcontroller and Microprocesso(BE05041011)

- Size: one byte
- Addressing: register
- Machine Cycle: 2 (F,MR)
- T-state: 7

CPI 8-bit [Compare 8-bit data with accumulator. Content of accumulator remain same]

- Size: two bytes
- Addressing: register
- Machine Cycle: 2 (F,MR)
- T-state: 7

RAL [Rotate accumulator left through carry means 7th bit goes to carry and carry goes to 0th bit position]

- Size: one byte
- Addressing: implied
- Machine Cycle: 1 (F)
- T-state: 4

RLC [Rotate accumulator left without carry means 7th bit goes to 0th bit position]

- Size: one byte
- Addressing: implied
- Machine Cycle: 1 (F)
- T-state: 4

RAR [Rotate accumulator right through carry means 0th bit goes to carry & goes to 7th bit position]

- Size: one byte
- Addressing: implied
- Machine Cycle: 1 (F)
- T-state: 4

RRC [Rotate accumulator right without carry means 0th bit goes to goes to 7th bit position]

- Size: one byte
- Addressing: implied
- Machine Cycle: 1 (F)
- T-state: 4

Instruction related to arithmetic & logical operation:

Sr. No	Arithmetic instruction		Logical instruction	
	OPCODE	OPERAND	OPCODE	OPERAND
1	ADD	R	ANA	R
2	ADD	M	ANI	8-BIT
3	ADI	8-BIT	ORA	R
4	SUB	R	ORI	8-BIT
5	SUB	M	XRA	R
6	SUI	8-BIT	XRI	8-BIT
7	INR	R	CMA	
8	INR	M	CMC	
9	DCR	R	STC	
10	DCR	M		
11	INX	Rp		
12	DCX	Rp		
13	ADC	R		
14	ADC	M		
15	ACI	8 BITS		
16	SBB	R		
17	SBB	M		
18	SBI	8 BITS		
19	DAD	Rp		
20	DAA			
21	STC			

Assembly language programs:

1. Write an ALP to add two 8-bit numbers stored in registers A and B. Store the result in register C. (Assume there is no carry).

Microcontroller and Microprocesso(BE05041011)

```
MVI A,04H
MVI B,18H
ADD B
MOV C,A
HLT
```

Registers

A/PSW0x1C02

BC0x181C

DE0x0000

HL0x0000

SP0xFFFF

PC0x0800

Flags

Z☐

S☐

P☐

C☐

AC☐

main.asm

1MVI A,04H

2MVI B,18H

3ADD B

4MOV C,A

5

6HLT

7

Assembler Output

13E04MVI A,04H

20618MVI B,18H

388ADD B

44FMOV C,A

5

676HLT

7

Load at 0x0800

Memory View

0x

C100H

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C11	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C12	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C13	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C14	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C15	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C16	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C17	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C18	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C19	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at 0x:BB00

Microcontroller and Microprocesso(BE05041011)

2. Write an ALP to subtract an 8-bit number in register B from an 8-bit number in register C. Store the result in register D (Assume there is no borrow).

MVI A,18H

MVI B,04H

SUB B

MOV C,A

HLT

The screenshot shows an 8085 assembly simulator interface. The **Registers** panel on the left displays the current state: A/PSW is 0x1416, BC is 0x0414, DE is 0x0000, HL is 0x0000, SP is 0xFF FF, and PC is 0x0800. The **Flags** panel shows Z=0, S=0, P=1 (checked), C=0, and AC=1 (checked). The **main.asm** code window contains the following instructions: 1. MVI A,18H, 2. MVI B,04H, 3. SUB B, 4. MOV C,A, 5. (blank), 6. HLT. The **Assembler Output** window shows the assembly process: 1. 3E 18 MVI A,18H, 2. 06 04 MVI B,04H, 3. 90 SUB B, 4. 4F MOV C,A, 5. (blank), 6. 76 HLT. The **Memory View** on the right shows a memory dump starting at address 0x0800, with all cells containing 00.

3. Write an ALP to a) Add b) Subtract two 8-bit numbers stored at memory locations C100H and C101H.

LDA C100H

MOV B,A

LDA C101H

ADD B

MOV D,A

LDA C101H

SUB B

MOV E,A

HLT

The screenshot shows an 8085 assembly simulator interface. The **Registers** panel displays: A/PSW is 0x1416, BC is 0x0400, DE is 0x1C14, HL is 0x0000, SP is 0xFF FF, and PC is 0x0800. The **Flags** panel shows Z=0, S=0, P=1 (checked), C=0, and AC=1 (checked). The **main.asm** code window contains: 1. LDA C100H, 2. MOV B,A, 3. LDA C101H, 4. ADD B, 5. MOV D,A, 6. LDA C101H, 7. SUB B, 8. MOV E,A, 9. (blank), 10. HLT. The **Assembler Output** window shows: 1. 3A 00 c1 LDA C100H, 2. 47 MOV B,A, 3. 3A 01 c1 LDA C101H, 4. 80 ADD B, 5. 57 MOV D,A, 6. 3A 01 c1 LDA C101H, 7. 90 SUB B, 8. 5F MOV E,A, 9. (blank), 10. 76 HLT. The **Memory View** on the right shows a memory dump starting at address 0x0800. Address C100H contains 15 (0x0F) and address C101H contains 15 (0x0F). All other memory cells contain 00.

Microcontroller and Microprocesso(BE05041011)

4. Write an ALP to shift the contents of register B by one bit using ADD instruction.

MVI A,05H

ADD A

MOV B,A

HLT

The screenshot shows an 8085 assembly simulator. The Registers window displays: A/PSW: 0x0A06, BC: 0x0A00, DE: 0x0000, HL: 0x0000, SP: 0xFF FF, PC: 0x0800. The Flags window shows: Z: ☐, S: ☐, P: ☒, C: ☐, AC: ☐. The Assembler Output window shows the following instructions: 1: 3E 05 MVI A,05H; 2: 87 ADD A; 3: 47 MOV B,A;SHIFTED NUMBER WAS STORED IN B; 4: HLT. The Memory View window shows the memory contents, with the value 05H stored at address 0000.

5. Write an ALP to store 16-bit data in register pairs BC and DE, add the two and store the result in memory locations C100H and C101H. (MSB at C101H)

a) Using DAD instruction

LXI B,1234H

LXI D,1234H

MOV H,B

MOV L,C

DAD D

MOV A,H

STA C101H

MOV A,L

STA C100H

HLT

The screenshot shows an 8085 assembly simulator. The Registers window displays: A/PSW: 0x6802, BC: 0x1234, DE: 0x1234, HL: 0x2468, SP: 0xFF FF, PC: 0x0800. The Flags window shows: Z: ☐, S: ☐, P: ☐, C: ☐, AC: ☐. The Assembler Output window shows the following instructions: 1: 01 34 12 LXI B,1234H; 2: 11 34 12 LXI D,1234H; 3: 68 MOV H,B; 4: 69 MOV L,C; 5: 19 DAD D; 6: 7C MOV A,H; 7: 32 01 c1 STA C101H; 8: 7D MOV A,L; 9: 32 00 c1 STA C100H; 10: HLT. The Memory View window shows the memory contents, with the value 1234H stored at address C100H and 1234H stored at address C101H.

Microcontroller and Microprocesso(BE05041011)

b) Without using DAD instruction

LXI B,1234H

LXI D,1234H

MOV A,D

ADD B

STA C101H

MOV A,E

ADD C

STA C100H

HLT

Registers 

A/PSW 0x 68 02

BC 0x 12 34

DE 0x 12 34

HL 0x 00 00

SP 0x FF FF

PC 0x 08 00

Flags 

Z ☐

S ☐

P ☐

C ☐

AC ☐

main.asm

```

1 LXI B,1234H
2 LXI D,1234H
3 MOV A,D
4 ADD B
5 STA C101H
6 MOV A,E
7 ADD C
8 STA C10BH
9
10 HLT

```

Assembler Output

```

1      01 34 12      LXI B,1234H
2      11 34 12      LXI D,1234H
3      7A             MOV A,D
4      8B             ADD B
5      32 01 c1       STA C101H
6      7B             MOV A,E
7      81             ADD C
8      32 0B c1       STA C10BH
9
10     76             HLT

```

Memory View 

0x C100H

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C10	88	24	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C11	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C12	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C13	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C14	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C15	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C16	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C17	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C18	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C19	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at: 0x BB00

6. It is required to multiply a number stored in accumulator by 3. Write an ALP for this application.

MVI A,03H

MOV C,A

MVI B,02H

BACK:ADD C

DCR B

JNZ BACK

HLT

Registers

A/PSW 0x0956

BC 0x0003

DE 0x0000

HL 0x0000

SP 0xFF FF

PC 0x0800

Flags

Z ☒

S ☐

P ☒

C ☐

AC ☒

main.asm

```

1  MVI A,03H
2  MOV C,A
3  MVI B,02H
4  BACK:ADD C
5  DCR B
6  JNZ BACK
7
8  HLT

```

Assembler Output

```

1  3E 03          MVI A,03H
2  4F             MOV C,A
3  06 02          MVI B,02H
4  81             BACK:ADD C
5  05             DCR B
6  C2 05 08       JNZ BACK
7
8  76             HLT

```

Load at 0x0800

Memory View

0x Address

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C11	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C12	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C13	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C14	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C15	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C16	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C17	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C18	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C19	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C1F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at: 0x

Microcontroller and Microprocessor(BE05041011)

- Write ALP to load data from memory location 2000H into the accumulator, performs a logical NOT (complement) operation on the accumulator, sets the carry flag to indicate the operation.

LDA 2000H

CMA

STC

HLT

Registers

A/PSW 0x AA 03

BC 0x 00 00

DE 0x 00 00

HL 0x 00 00

SP 0x FF FF

PC 0x 08 05

Flags

Z ☐

S ☐

P ☐

C ☒

AC ☐

main.asm

```
1 LDA 2000H ; Load data from memory location 2000H into the accumulator
2 CMA ; Perform logical NOT operation (complement) on the accumulator
3 STC ; Set the carry flag
4 HLT ; Halt the program
5
```

Memory View

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2000	3A	00	20	00	00	00	00	00	00	00	00	00	00	00	00	00
2001	2F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2002	37	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2003	76	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2004	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2005	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2006	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2007	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2008	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2009	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
200A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
200B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
200C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
200D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
200E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
200F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Assembler Output

```
1 3A 00 20 LDA 2000H ; Load data from memory location 2000H into accumulator
2 2F 00 00 CMA ; Perform logical NOT operation (complement) on the accumulator
3 37 00 00 STC ; Set the carry flag
4 76 00 00 HLT ; Halt the program
5
```

- Write a ALP to load the values A2H and 04H into the accumulator and register B, respectively. Then, perform a logical OR operation.

MVI A, 0A2H

MVI B, 04H

ORA B

HLT

Registers

A/PSW 0x A6 B6

BC 0x 04 00

DE 0x 00 00

HL 0x 00 00

SP 0x FF FF

PC 0x 08 05

Flags

Z ☐

S ☒

P ☒

C ☐

AC ☐

main.asm

```
1 MVI A, 0A2H ; Load hexadecimal value A2H into the accumulator
2 MVI B, 04H ; Load hexadecimal value 04H into register B
3 ORA B ; Perform logical OR operation between the accumulator and B
4 HLT ; Halt the program
5
```

Memory View

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0001	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0002	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0003	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0004	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0005	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0006	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0007	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0008	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0009	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Assembler Output

```
1 3E a2 MVI A, 0A2H ; Load hexadecimal value A2H into the accumulator
2 06 04 MVI B, 04H ; Load hexadecimal value 04H into register B
3 B0 00 00 ORA B ; Perform logical OR operation between the accumulator and B
4 76 00 00 HLT ; Halt the program
5
```

Safety and necessary Precautions:

- Do not start the computer or kit without instructions

Conclusion:

Quiz:

1. What is the purpose of the ADD instruction in 8085 assembly language programming?
2. Which 8085 instruction is used for logical AND operations?
3. In 8085 assembly language, which instruction is used to decrement the contents of a register or memory location?
4. What is the role of the CMP instruction in 8085 programming?
5. Which 8085 instruction is used to clear (set to 0) a specific bit in a register or memory location?
6. What is the purpose of the XRA instruction in 8085 assembly language programming?
7. Which instruction is used in 8085 assembly language to perform addition with carry?

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

1. Instruction Proficiency: Evaluate the ability to correctly implement arithmetic and logical instructions in assembly language, focusing on accuracy and precision.
2. Algorithm Efficiency: Assess the efficiency of the algorithms developed, considering the use of arithmetic and logical instructions to manipulate data effectively.
3. Logical Flow: Evaluate the logical flow of the assembly code, ensuring that the sequence of instructions aligns with the desired data manipulation.
4. Error Minimization: Measure the accuracy of the code, emphasizing the reduction of errors in arithmetic and logical operations.
5. Problem Solving: Evaluate problem-solving skills in the context of using arithmetic and logical instructions to address specific computational tasks.

Experiment No: 4

Assembly language programming using branching and looping instructions.

Date:

Competency and Practical Skills: Competency in assembly language programming with branching and looping instructions will be achieved, along with practical skills for implementing such instructions effectively.

Relevant CO: CO2

Objectives: To develop proficiency in assembly language programming by effectively employing branching and looping instructions.

Equipment/Instruments/Software: <https://www.sim8085.com>. However, other software or microprocessor kits or educational boards may also be used.

Theory:

The branching instructions allow the microprocessor to change the sequence of a program, either conditionally or under certain conditions.

These are two different types of branching operation performed in Microprocessor –

- 1) Unconditional branching
- 2) Conditional branching

In conditional branching operation, program execution will be transferred from current location to another location according to condition of flag which is changed after Arithmetic & Logical operation.

In unconditional branching operations program will be transferred from one place to another place irrespective to flag condition.

There are again two types in conditional & Unconditional branching: JMP & CALL. In case of JMP instruction, once the program will go to other location it will not come to the same location where it leaves. In case of CALL instruction, after performing subroutine operation program will come again to main routine.

Similarly, there are two types of return instructions: Unconditional & Conditional. Using return instruction program return from the subroutine program.

The instructions for branching operation are listed below:

Unconditional Branching:

CALL Add [Call to the given address]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 18

JMPAdd [Jump to the given address]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 10

Conditional Branching:

CC Add [Call to the given address if carry is generated]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 9/18

CM Add [Call to the given address if result is negative]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 9/18

CNC Add [Call to the given address if there is no carry]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 9/18

CNZ Add [Call to the given address if result is not zero]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 9/18

CP [Call to the give address if result is positive]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 9/18

CPE [Call to the given address if even parity is set]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 9/18

CPO [Call to the given address if odd parity is set]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 9/18

CZ [Call to the given address if result is zero]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 5 (F,MR,MR,MW,MW)
- T-state: 9/18

RET [Return from subroutine]

- Size: one byte
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 6/12

RZ [Return from the subroutine if result is not zero]

- Size: one byte
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 6/12

RNZ [Return from the subroutine if result is not zero]

- Size: one byte
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 6/12

RC [Return from the subroutine if carry is generated]

- Size: one byte

Microcontroller and Microprocesso(BE05041011)

- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 6/12

RNC [Return from the subroutine if carry is not generated]

- Size: one byte
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 6/12

RM [Return from the subroutine if result is negative]

- Size: one byte
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 6/12

RP [Return from the subroutine if result is positive]

- Size: one byte
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 6/12

JZ [Jump to given address if result is zero]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 7/10

JNZ [Jump to the given address if result is not zero]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 7/10

JC [Jump to the given address if carry is generated]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 7/10

JNC [Jump to the given address if carry is not generated]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 7/10

JM [Jump to the given address if result is negative]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 7/10

JP [Jump to the given address if result is positive]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 7/10

JPE [Jump to the given address if result contain even parity]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 7/10

JPO [Jump to the given address if result contain if odd parity]

- Size: three bytes
- Addressing: direct
- Machine Cycle: 3 (F,MR,MR)
- T-state: 7/10

Instructions related to branching & looping operation:

Sr. No	Branching & looping instruction	
	OPCODE	OPERAND
1	JMP	16 –bit
2	JZ	16 –bit
3	JNZ	16 –bit
4	JC	16 –bit
5	JNC	16 –bit
6	JP	16 –bit
7	JM	16 –bit

Microcontroller and Microprocesso(BE05041011)

8	JPE	16-bit
9	JPO	16-bit

Assembly language programs:

1. Write down an ALP which find largest of two no and store the result in memory location C200H.

```
LDA C210H
MOV B,A
LDA C211H
CMP B
JNC L1
MOV A,B
STA C200H
L1:STA C200H
HLT
```

Registers

A/PSW	0x0893
BC	0x0800
DE	0x0000
HL	0x0000
SP	0xFF FF
PC	0x0800

Flags

Z	☐
S	☒
P	☐
C	☒
AC	☒

Memory View

Load at 0x0800

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C20	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C21	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C22	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C23	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C24	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C25	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C26	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C27	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C28	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C29	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C2A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C2B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C2C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C2D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C2E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C2F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at: 0x BB00

Assembler Output

1	3A 10 C2	LDA C210H
2	47	MOV B,A
3	3A 11 C2	LDA C211H
4	B8	CMP B
5	D2 0F 08	JNC L1
6	78	MOV A,B
7	32 00 C2	STA C200H
8	32 00 C2	L1:STA C200H
9		
10	76	HLT

2. Write an ALP to find out maximum number in block of data. The block starts at E000H. The first data show length of block. Store the result at the end of the block.

```
LXI H,E000H
MOV C,M
MOV E,C
INR E
MOV D,H
INX H
MOV B,M
DCR C
LOOP:INX H
MOV A,M
CMP B
JC SKIP
MOV B,A
SKIP:DCR C
JNZ LOOP
MOV A,B
```

Registers

A/PSW	0x: 80 57
BC	0x: 80 00
DE	0x: E0 08
HL	0x: D0 07
SP	0x: FF FF
PC	0x: 08 00

Flags

Z	☒
S	☐
P	☒
C	☒
AC	☒

Load at 0x0800

Memory View

0x E000H

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
E00	07	75	48	25	80	35	14	59	80	00	00	00	00	00	00	00
E01	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E02	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E03	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E04	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E05	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E06	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E07	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E08	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E09	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E0A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E0B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E0C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E0D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E0E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E0F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at 0x: DD00 ▾

main.asm

```

1  LXI  H,E000H
2  MOV  C,H
3  MOV  E,C
4  INR  E
5  MOV  D,H
6  INX  H
7  MOV  B,H
8  DCR  C
9  LOOP:INX  H
10 MOV  A,H
11 CMP  B
12 JC  SKIP
13 MOV  B,A
14 SKIP:DCR  C
15 JNZ  LOOP
16 MOV  A,B
17 STAX  D
18
19 HLT

```

Assembler Output

1	21 00 e0	LXI H,E000H
2	4E	MOV C,H
3	59	MOV E,C
4	1C	INR E
5	54	MOV D,H
6	23	INX H
7	46	MOV B,H
8	00	DCR C
9	23	LOOP:INX H
10	7E	MOV A,H
11	88	CMP B
12	DA 11 08	JC SKIP
13	47	MOV B,A
14	80	SKIP:DCR C
15	C3 00 08	JNZ LOOP

Microcontroller and Microprocesso(BE05041011)

STAX D

HLT

3. Write an ALP to add five data bytes stored in memory location starting at C160. Store the result in C200H (LSB) and C201H(MSB).

LXI H,C160H

MVI C,05H

MVI D,00H

MOV A,M

AGAIN:INX H

MOV B,M

ADD B

JNC L1

INR D

L1:DCR C

JNZ AGAIN

STA C200H

MOV A,D

STA C201H

HLT

The screenshot displays an 8085 assembly simulator interface. It includes a 'Registers' panel on the left showing A/PSW (0x0156), BC (0x0000), DE (0x0100), HL (0xC165), SP (0xFFFF), and PC (0x0800). Below it, the 'Flags' panel shows Z (checked), S (unchecked), P (checked), C (unchecked), and AC (checked). The 'Memory View' on the right shows a hex dump of memory starting at C160, with values 46, 64, 49, 67, 37 at addresses C160 through C164. The 'Assembler Output' at the bottom shows the assembly code being executed, including instructions like LXI H,C160H, MVI C,05H, MVI D,00H, MOV A,M, AGAIN:INX H, MOV B,M, ADD B, JNC L1, INR D, L1:DCR C, JNZ AGAIN, STA C200H, MOV A,D, STA C201H, and HLT.

4. A set of eight data byte is stored in the memory location starting at memory location 1000H. Check each data byte for bits D7 and bit D0. If D7 or D0 is 1, reject the data byte otherwise store the data byte at memory location starting at 1010H.

Microcontroller and Microprocesso(BE05041011)

LXI B,1000H
LXI D,1010H
MVI L,08H
BACK:LDAX B
ANI 81H
STAX D
LDAX B
LDAX D
INX B
INX D
DCR L
JNZ BACK
HLT

The screenshot displays an 8085 assembly simulator interface. On the left, the **Registers** panel shows A/PSW at 0x0156, BC at 0x1008, DE at 0x1018, HL at 0xE000, SP at 0xFFFF, and PC at 0x0800. The **Flags** panel shows Z, S, P, C, and AC flags. The **Assembly Output** panel lists 15 instructions with their corresponding hex codes and addresses. The **Memory View** panel shows a memory dump starting at 0x0000, with the first few bytes containing the program code.

Register	Value
A/PSW	0x0156
BC	0x1008
DE	0x1018
HL	0xE000
SP	0xFFFF
PC	0x0800

Flag	Status
Z	☒
S	☐
P	☒
C	☐
AC	☒

Address	Hex	Assembly
0100	01 00 10	LXI B,1000H
0103	11 10 10	LXI D,1010H
0106	2E 08	MVI L,08H
0109	0A	BACK:LDAX B
010A	E6 81	ANI 81H
010D	12	STAX D
010E	0A	LDAX B
010F	1A	LDAX D
0110	03	INX B
0111	13	INX D
0112	2D	DCR L
0113	C2 08 08	JNZ BACK
0116	76	HLT

5. Calculate the sum of series of even numbers from the given list of numbers. The length of the list is in memory location 2200H and the series begins from memory location 2201H. Result will store at memory location 2210H.

Input :

2200H= 4H

2201H= 20H

2202H= 15H

2203H= 13H

2204H= 22H

Output :

Result 2210H = 42H

LDA 2200H
MOV C, A
MVI B, 00H
LXI H, 2201H
BACK: MOV A, M
ANI 01H
JNZ SKIP
MOV A, B
ADD M
MOV B, A
SKIP: INX H

The screenshot displays an 8085 assembly simulator interface. On the left, the **Registers** panel shows A/PSW at 0x4256, BC at 0x4200, DE at 0x0000, HL at 0x2205, SP at 0xFFFF, and PC at 0x081C. The **Flags** panel shows Z, S, P, C, and AC flags. The **Assembly Output** panel lists 17 instructions with their corresponding hex codes and addresses. The **Memory View** panel shows a memory dump starting at 0x2200, with the first few bytes containing the program code.

Register	Value
A/PSW	0x4256
BC	0x4200
DE	0x0000
HL	0x2205
SP	0xFFFF
PC	0x081C

Flag	Status
Z	☒
S	☐
P	☒
C	☐
AC	☒

Address	Hex	Assembly
2200	04 20 15 13 22	LDA 2200H
2201	42	MOV C, A
2202	00	MVI B, 00H
2203	00	LXI H, 2201H
2204	00	BACK: MOV A, M
2205	00	ANI 01H
2206	00	JNZ SKIP
2207	00	MOV A, B
2208	00	ADD M
2209	00	MOV B, A
220A	00	SKIP: INX H
220B	00	DCR C
220C	00	JNZ BACK
220D	00	MOV A, B
220E	00	STA 2210H
220F	00	HLT

Microcontroller and Microprocesso(BE05041011)

```
DCR C
JNZ BACK
MOV A, B
STA 2210H
HLT
```

Assembler Output		
1	3A 00 22	LDA 2200H
2	4F	MOV C, A
3	06 00	MVI B, 00H
4	21 01 22	LXI H, 2201H
5	7E	BACK: MOV A, M
6	E6 01	ANI 01H
7	C2 12 08	JNZ SKIP
8	78	MOV A, B
9	86	ADD M
10	47	MOV B, A
11	23	SKIP: INX H
12	0D	DCR C
13	C2 09 08	JNZ BACK
14	78	MOV A, B
15	32 10 22	STA 2210H

Microcontroller and Microprocesso(BE05041011)

6. Statement: Multiply two 8-bit numbers stored in memory locations 2200H and 2201H by repetitive addition and store the result in memory locations 2210H and 2211H.

Sample problem:

$$(2200H) = 03H \& (2201H) = B2H$$

$$\text{Result} = B2H + B2H + B2H = 0216H$$

$$(2210H) = 16H \& (2211H) = 02H$$

```

LDA 2200H
MOV E, A
MVI D, 00
LDA 2201H
MOV C, A
LXI H, 0000 H
BACK: DAD D
DCR C
JNZ BACK
SHLD 2300H
HLT

```

The screenshot displays an 8085 assembly simulator interface. On the left, the **Registers** panel shows: A/PSW at 0xB256, BC at 0x0000, DE at 0x0003, HL at 0x0216, SP at 0xFFFF, and PC at 0x0816. Below it, the **Flags** panel shows Z (checked), S (unchecked), P (checked), C (unchecked), and AC (checked). The central **main.asm** window lists 12 instructions: 1. LDA 2200H, 2. MOV E, A, 3. MVI D, 00, 4. LDA 2201H, 5. MOV C, A, 6. LXI H, 0000H, 7. BACK: DAD D, 8. DCR C, 9. JNZ BACK, 10. SHLD 2300H, 11. HLT, and 12. (blank). On the right, the **Memory View** shows a hex dump starting at address 2200, with the first two bytes (03, B2) highlighted in green, corresponding to the first instruction.

Safety and necessary Precautions:

1. Do not start the computer or kit without instructions

Conclusion:

Quiz:

1. What is the difference between conditional & unconditional branching instruction?
2. What is the difference in operation of JMP & CALL instruction?

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

1. Execution Efficiency: Assess the effectiveness and efficiency of the program in utilizing branching and looping instructions to achieve the desired functionality.
2. Logical Flow and Control: Evaluate the clarity and accuracy of the program's logic in implementing branching and looping structures.
3. Error Handling: Examine the ability to address and manage potential errors that may arise during the execution of branching and looping instructions.
4. Code Optimization: Measure the degree to which the code maximizes the utilization of branching and looping instructions, minimizing redundancy.

5. Overall Proficiency: Determine the overall competence in assembly language programming when using branching and looping instructions, considering both technical proficiency and problem-solving skills.

INTRODUCTION

Earlier to Microcontrollers, Microprocessors were greatly used for each and every purpose. Microprocessors were containing ALU, general purpose register, stack pointer, program counter, clock counter and so many other features which today's Microcontroller also possesses. But the difference between them exists with respect to the number of instructions, access times, size, reliability, PCB size, and so on. The microprocessor contains a large instruction set called as CISC processor whereas Microcontroller contains less number of instructions and is called as RISC processor. The access time is less in the case of microcontrollers compared to microprocessors and the PCB size reduces in the case of microcontrollers.

There are many versions of microcontrollers 8051, 8052, 8751, AT8951 from Atmel Corporation, and many more. In this manual, we will study the 8051 architecture, its features, programming, and interfacing.

MCS 8051 is an 8-bit single-chip microcontroller with many built-in functions and is the core for all MCS-51 devices.

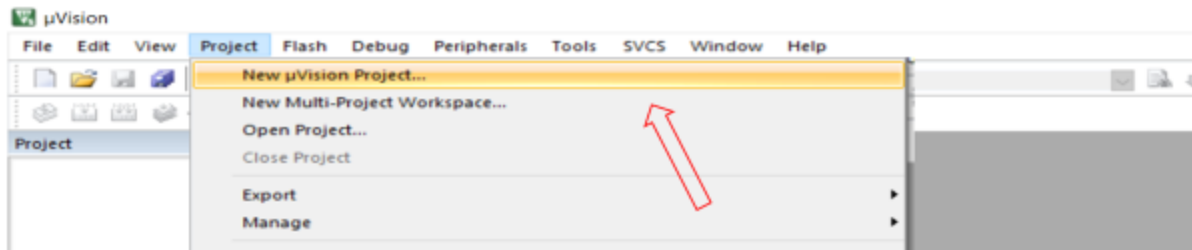
The main features of the 8051 core are:

- Operates with single Power Supply +5V.
- 8-bit CPU optimized for control applications.
- 16-bit program counter (PC) and 16-bit data pointer (DPTR).
- 8-bit program status word (PSW).
- 8-bit stack pointer (SP).
- 4K Bytes of On-Chip Program Memory (Internal ROM or EPROM).
- 128 bytes of On-Chip Data Memory (Internal RAM):
- Four Register Banks, each containing 8 registers (R0 to R7) [Total 32 reg]
- 16-bytes of bit addressable memory.
- 80 bytes of general-purpose data memory (Scratch Pad Area).
- Special Function Registers (SFR) to configure/operate microcontroller.
- 32 bit bi-directional I/O Lines (4 ports P0 to P3).
- Two 16-bit timers/counters (T0 and T1).
- Full duplex UART (Universal Asynchronous Receiver/Transmitter).
- On-Chip oscillator and clock circuitry

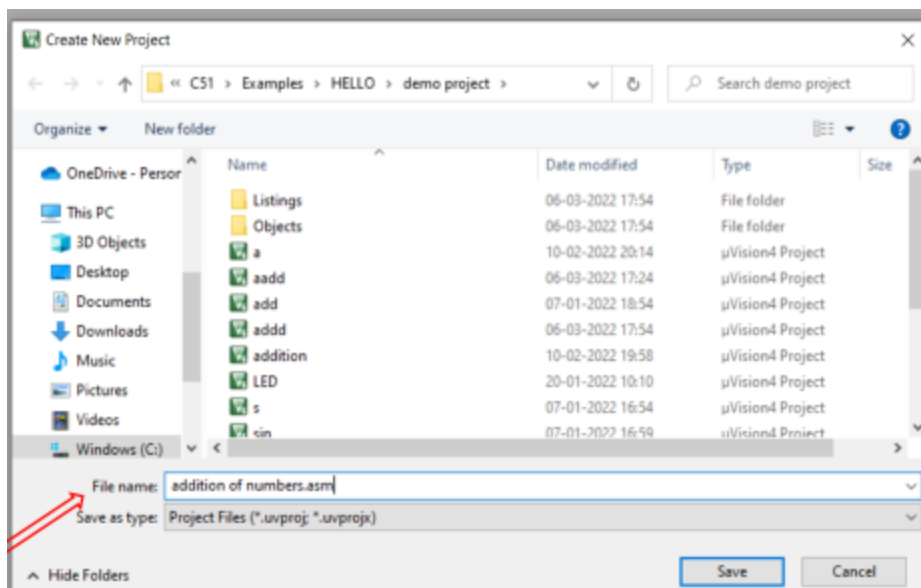
Executing assembly language programs in Keil Microvision Software:

Follow the following steps to execute assembly language programs in Keil Microvision software:

Step 1: Open Keil Microvision software then click on the project then click on the new Microvision project.



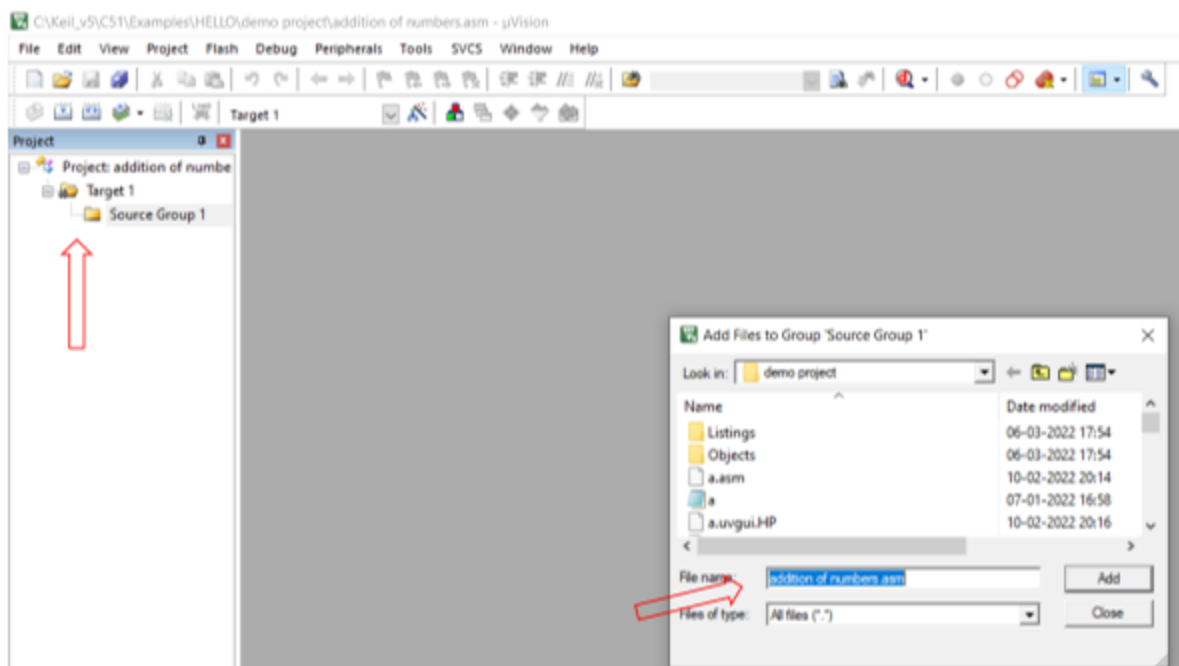
Step 2: Save your file with .asm extension for example (addition of numbers.asm) and save it.



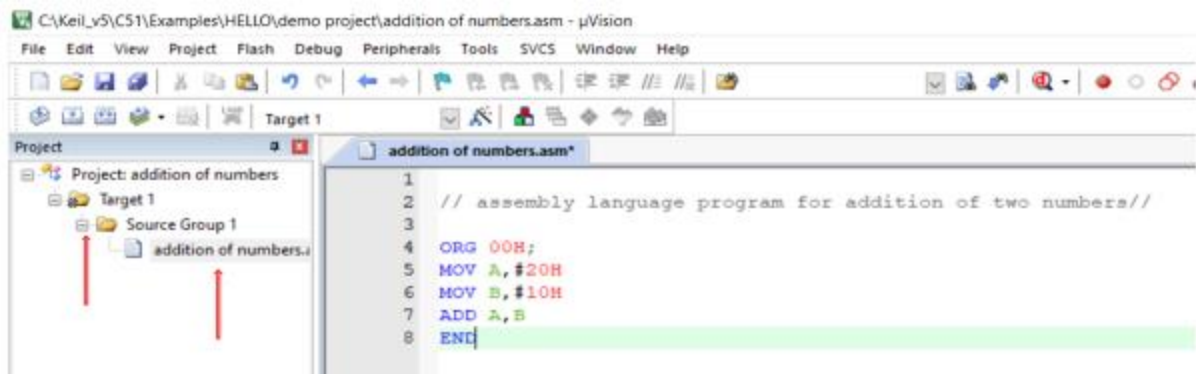
Step 3: Select microchip AT89C51 and click on ok.



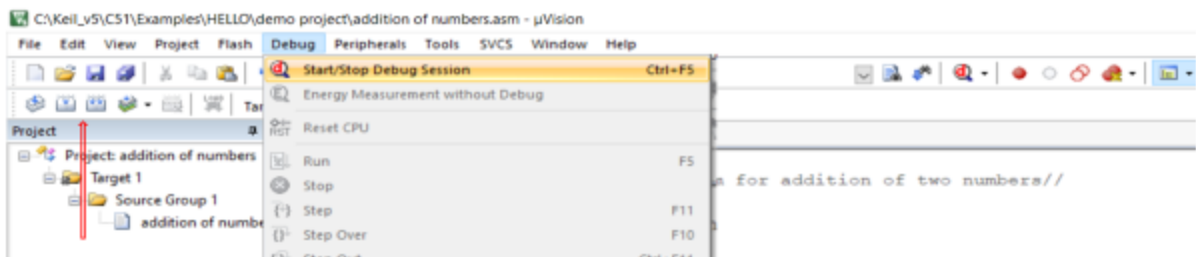
Step 4: Now click on target 1 and then double click on source group 1, select the file that you saved previously with extension **.asm** add it to the source group, and close it.



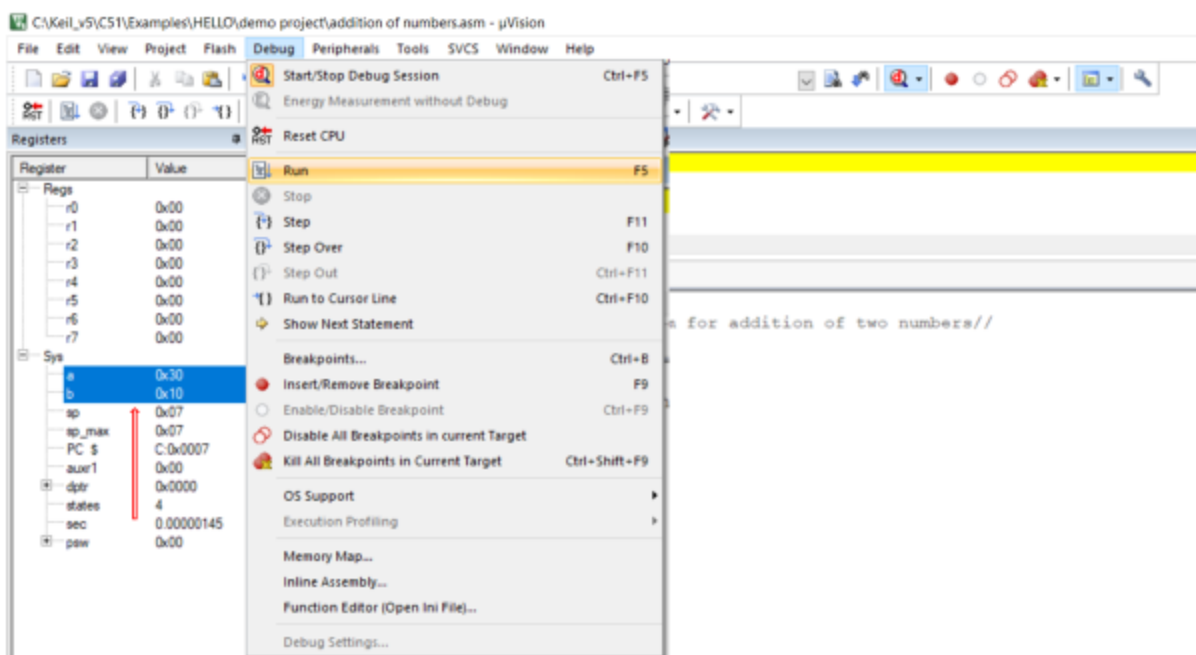
Step 5: Now click on the + sign shown before source group 1, then double click on the file that comes after clicking on +sign. Then write assembly language code in the plane text area that appears.



Step 6: Click on build and rebuild to check errors then click on debug and start debug session.



Step 7: Then click on run you can run directly or you can run step by step by clicking every time on the step or pressing F11 and then check the output here the output is visible in blue color.



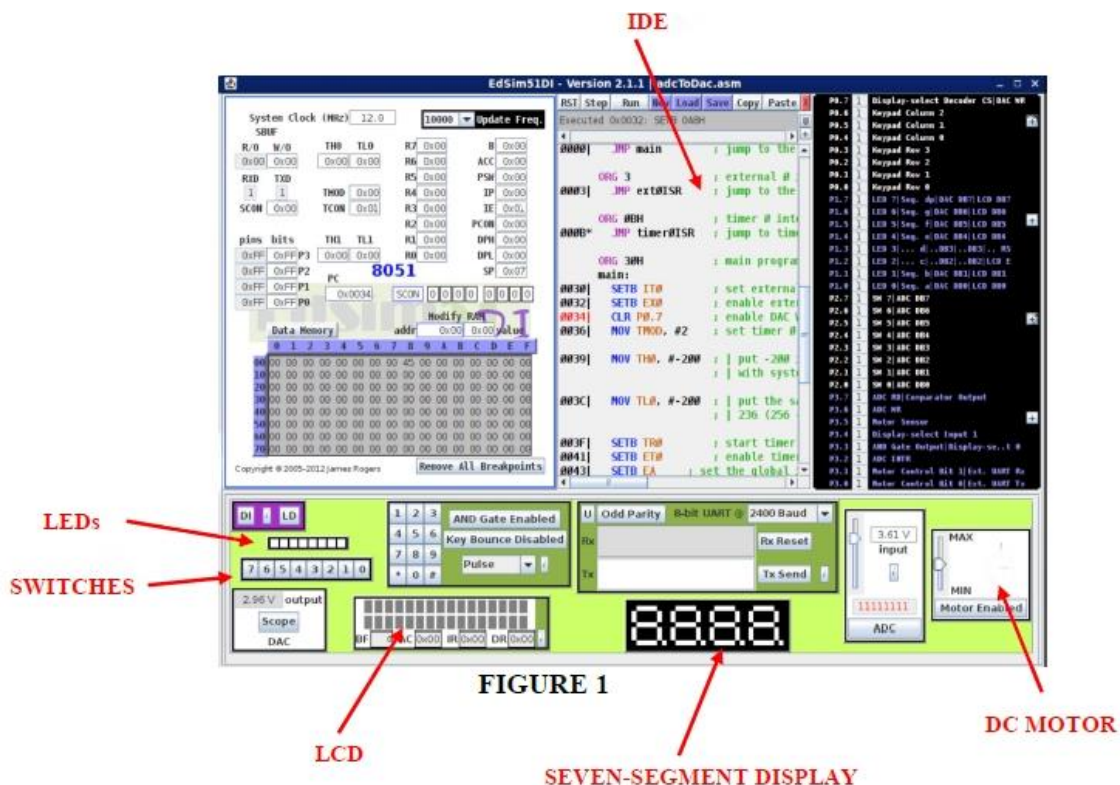
By following the above-mentioned steps, you can execute any type of assembly language statements or instructions and assembler directives in Keil Microvision software.

Familiarization with the EdSim51 simulator :

EdSim51 is a free, web-based, educational software package that is used to simulate and analyze 8051-based microcontroller systems. It is designed to provide students with a hands-on learning experience and is a useful tool for instructors teaching microcontroller-based courses.

Students can use EdSim51 to design, code, and simulate programs on a virtual 8051 microcontroller, allowing them to test and debug their programs in a safe and controlled environment. Additionally, EdSim51 provides a variety of built-in peripherals, such as LCDs, keypad input, and serial communication, allowing students to design and test complete systems.

One of the key benefits of EdSim51 is that it is accessible and user-friendly. It can be accessed from any computer with an internet connection, and it does not require any additional software or hardware. This makes it an ideal tool for remote learning and for students who do not have access to physical microcontroller systems.



The top left box is the microcontroller panel which gives the student access to all of the 8051's registers, data memory, and code memory. The upper middle part is the integrated development environment (IDE) or the assembly code panel where the user either loads an assembly program or writes the code directly. The top right box displays the list of the 32 port pins, the current value of the port pin, and what each is connected to.

Overall, EdSim51 is a valuable resource for students learning about microcontrollers and embedded systems, providing them with a powerful tool for experimentation, learning, and project development.

Experiment No: 5

To study and perform data transfer programming.

Date:

Competency and Practical Skills: Ability to perform arithmetic operations on 8-bit and 16-bit numbers using microcontrollers and proficiency in using programming languages and software tools required for microcontroller-based systems.

Relevant CO: CO1, CO2

Objectives: Gain proficiency in programming and performing arithmetic operations on microcontrollers.

Equipment/Instruments/Software: Keil Microvision software or EdSim51. However, other software or microcontroller development boards or educational boards may also be used.

Sample Programs:

1. Write an assembly language program to transfer N = _____ bytes of data from location A: _____ h to location B: _____ h.

Let N = 05h, A: 30h and B: 40h

Label	Operands	Remarks
BACK:	MOV R0,#30H MOV R1,#40H MOV R7,#05H MOV A,@R0 MOV @R1,A INC R0 INC R1 DJNZ R7,BACK END	; source address ; destination address ; Number of bytes to be moved ; repeat till all data transferred

2. Write an assembly language program to exchange N = ____ h bytes of data at location A: ____ h and at location B: ____ h.

Let N = 05h, A: 30h and B: 40h

Label	Operands	Remarks
BACK:	MOV R0,#30H MOV R1,#40H MOV R7,#05H MOV A,@R0 MOV R4,A MOV A,@R1 MOV @R0,A MOV A,R4 MOV @R1,A INC R0 INC R1 DJNZ R7,BACK END	; source address ; destination address ; count, the number of data to be exchanged

Exercises:

1. Write an assembly language program to find the largest element in a given array of N = ___h bytes at location 4000h. Store the largest element at location 4062h.
2. Write an assembly language program to sort an array of N = ___h bytes of data in ascending/descending order stored from location 9000h. (Using bubble sort algorithm)
3. Write an ALP in 8051 to Find maximum of the numbers stored in internal memory block from 40h to 5fh, send result to port 2 pins.
4. Write an ALP to Find minimum of the numbers stored in external RAM locations 2000h to 201fh, send result to external memory location 3000h.
5. Write a program in 8051 to sort data of memory location block in internal RAM locations 20h to 2fh in ascending order.

Safety and necessary Precautions:

1. Do not start the computer without instructions

Procedure:

1. Open Keil uVision software and create a new project.
2. Select the target device and configure the project settings.
3. Write the program code in C or assembly language using the Keil IDE.
4. Build the program and check for errors or warnings.

Conclusion:**Quiz:**

1. List the available software for the microcontroller practical.
2. Describe the dissimilarities between internal memory and external memory in the 8051 microcontroller.

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

1. **Instruction Proficiency:**Evaluate the ability to correctly implement arithmetic and logical instructions in assembly language, focusing on accuracy and precision.
2. **Algorithm Efficiency:**Assess the efficiency of the algorithms developed, considering the use of arithmetic and logical instructions to manipulate data effectively.
3. **Logical Flow:**Evaluate the logical flow of the assembly code, ensuring that the sequence of instructions aligns with the desired data manipulation.
4. **Error Minimization:**Measure the accuracy of the code, emphasizing the reduction of errors in arithmetic and logical operations.
5. **Problem Solving:** Evaluate problem-solving skills in the context of using arithmetic and logical instructions to address specific computational tasks.

Experiment No: 6

To study and perform arithmetic instruction programming.

Date:

Competency and Practical Skills: Analyzing data transfer and arithmetic operations on external memory.

Relevant CO: CO1, CO2

Objectives: Gain a better understanding of data transfer and arithmetic operations on external memory, and investigate methods for optimizing their performance.

Equipment/Instruments/Software: Keil Microvision software or EdSim51. However, other software or microcontroller development boards or educational boards may also be used.

Sample Programs:

1. Write an assembly language program to perform the addition of two 16-bit numbers.

Label	Operands	Remarks
	MOV R0,#34H	; lower nibble of No.1
	MOV R1,#12H	; higher nibble of No.1
	MOV R2,#0DCH	; lower nibble of No.2
	MOV R3,#0FEH	; higher nibble of No.2
	CLR C	
	MOV A,R0	
	ADD A,R2	
	MOV 22H,A	
	MOV A,R1	
	ADDC A,R3	
	MOV 21H,A	
	MOV 00H,C	
	END	

2. Write an assembly language program to perform the subtraction of two 16-bit numbers.

Label	Operands	Remarks
	MOV R0,#0DCH	; lower nibble of No.1
	MOV R1,#0FEH	; higher nibble of No.1
	MOV R2,#34H	; lower nibble of No.2
	MOV R3,#12H	; higher nibble of No.2
	CLR C	
	MOV A,R0	
	SUBB A,R2	
	MOV 22H,A	
	MOV A,R1	
	SUBB A,R3	
	MOV 21H,A	

	MOV 00H,C END	
--	------------------	--

Exercises:

1. Write a program in 8051 assembly language to add numbers stored in internal memory block addressed from 30h to 3fh, store the result high byte to address of internal RAM 40h and low byte to 41h.
2. Write an assembly language program to find the square of a number stored at memory location 30h. Store the higher byte at 40h and lower byte at 41h.
3. Write a program in 8051 assembly language to add two numbers stored in external memory location 2000h and 2001h, store result in external memory location 3000h, also if answer contains carry, store the carry flag in LSB of external memory location 3001h.

Safety and necessary Precautions:

1. Do not start the computer without instructions

Procedure:

1. Open Keil uVision software and create a new project.
2. Select the target device and configure the project settings.
3. Write the program code in C or assembly language using the Keil IDE.
4. Build the program and check for errors or warnings.

Conclusion:

Quiz:

1. Describe the purpose of studying data transfer and arithmetic operations on external memory.
2. Define external memory.

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

1. Group work while performing experiment on computer.
2. Understand the concept while performing coding.
3. Understanding of data transfer instructions and arithmetic operations.
4. Debugging and testing skills.
5. Quiz answers and submission in time.

Experiment No: 7

To study and perform Boolean & logical instructions programming.

Date:

Competency and Practical Skills : Develop competencies and skills such as programming, problem-solving, debugging, familiarity with 8051 architecture.

Relevant CO: CO1, CO3

Objectives: To understand how to manipulate binary data and perform logical operations on it using the various instructions available in the 8051 microcontroller.

Equipment/Instruments/Software: Keil Microvision software or EdSim51. However, other software or microcontroller development boards or educational boards may also be used.

Sample Programs:

1. Write an ALP to compare two eight bit numbers NUM1 and NUM2 stored in external memory locations 8000h and 8001h respectively. Reflect your result as: If NUM1<NUM2, SET LSB of data RAM location 2FH (bit address 78H). If NUM1>NUM2, SET MSB of location 2FH (bit address 7FH). If NUM1 = NUM2, then Clear both LSB & MSB of bit addressable memory location 2FH.

Label	Operands	Remarks
	MOV DPTR,#8000H MOVX A,@DPTR MOV R0,A INC DPTR MOVX A,@DPTR CLR C SUBB A,R0 JZ EQUAL JNC SMALL SETB 7FH SJMP END1 SMALL: SETB 78H SJMP END1 EQUAL: CLR 78H CLR 7FH END1: END	 ; store external memory location ; pointer for external memory location ; subtract ; check for equal value and jump if yes ; check for lower value and jump if yes ; if not lower and equal then higher value

2. Write an assembly language program to count number of ones and zeros in a eight bit number.

Label	Operands	Remarks
	MOV R1,#00H	// to count number of 0s
	MOV R2,#00H	// to count number of 1s
	MOV R7,#08H	// counter for 8-bits
	MOV A,#97H	// data to count number of 1s and 0s
AGAIN:	RLC A	
	JC NEXT	
	INC R1	
NEXT:	SJMP HERE	
HERE:	INC R2	
	DJNZ R7,AGAIN	
	END	

Exercises:

1. Write a program in 8051 to sort data of memory location block in internal RAM locations 20h to 2fh in ascending order.
2. Write an assembly language program to find whether given eight bit number is odd or even. If odd store 00h in accumulator. If even store FFh in accumulator.
3. Write an assembly language program to perform logical operations AND, OR, XOR on two eight bit numbers stored in internal RAM locations 21h,22h.

Safety and necessary Precautions:

1. Do not start the computer without instructions

Procedure:

1. Open Keil uVision software and create a new project.
2. Select the target device and configure the project settings.
3. Write the program code in C or assembly language using the Keil IDE.
4. Build the program and check for errors or warnings.

Conclusion:

Quiz:

1. Explain the purpose of the ANL instruction in 8051 programming.
2. Illustrate how to clear a bit in a byte using the 8051 microcontroller.

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

1. Group work while performing experiment on computer.
2. Understand the concept while performing coding.
3. Understanding of logical instructions and algorithms.
4. Debugging and testing skills.
5. Quiz answers and submission in time.

Experiment No: 8

To study and perform LED Blinking Using 8051 Assembly Language and SimulIDE

Date: _____.

Competency and Practical Skills: Ability to understand 8051 I/O port operation, develop Assembly Language programs, interface an LED with a microcontroller, generate a HEX file, and use simulation software.

Relevant CO: CO1, CO2

Objectives: To write and test an 8051 Assembly Language program for LED blinking, generate a HEX file, interface the LED with P1.0, and simulate the circuit using SimulIDE.

Equipment/Instruments/Software: Keil μ Vision / Keil A51 or compatible 8051 assembler, SimulIDE, generated HEX file, AT89C51 or compatible 8051, LED, 330 Ω resistor, 11.0592 MHz crystal, two 33 pF capacitors, +5 V supply and GND, and reset circuit.

1. Aim

To write an 8051 Assembly Language program for blinking an LED connected to Port 1.0 (P1.0) and simulate the interfacing circuit and program operation using SimulIDE.

2. Objectives

- Understand 8051 I/O port operation.
- Write an 8051 Assembly Language program.
- Generate a HEX file for simulation.
- Interface an LED with P1.0.
- Create the circuit in SimulIDE and load the HEX file.
- Observe continuous LED blinking.
- Understand software delay using nested DJNZ loops.

3. Introduction

The 8051 is a popular 8-bit microcontroller family used in embedded-system education and applications. It provides four 8-bit I/O ports, timers/counters, serial communication, interrupt facilities, and an instruction set suitable for control applications.

In this experiment, an LED is connected to P1.0 of an AT89C51-compatible 8051. The Assembly program alternately writes logic 0 and logic 1 to P1.0, with a delay between the changes, producing continuous LED blinking.

4. Software and Hardware Requirements

Category	Requirements
Software	Keil μ Vision / Keil A51 or compatible 8051 assembler; SimulIDE; generated HEX file
Hardware	AT89C51 or compatible 8051; LED; 330 Ω resistor; 11.0592 MHz crystal; two 33 pF capacitors; +5 V supply and GND; reset circuit

5. Interfacing Connections

Pin/Component	Connection	Purpose
P1.0 (Pin 1)	330 Ω $\rightarrow$ LED $\rightarrow$ GND	LED output
XTAL1 (Pin 19)	11.0592 MHz crystal	Clock
XTAL2 (Pin 18)	11.0592 MHz crystal	Clock
Crystal capacitors	33 pF from each crystal side to GND	Oscillator stability
VCC	+5 V	Power
GND	0 V	Ground
EA (Pin 31)	+5 V	Internal program memory
RST (Pin 9)	Reset circuit	Reset

The LED is connected from P1.0 through a 330 Ω resistor to ground. Therefore, P1.0 = 1 turns the LED ON and P1.0 = 0 turns it OFF.

6. Interfacing Diagram for SimulIDE

[Interfacing diagram from source document]

Figure 1: 8051 LED-blinking interfacing concept for SimulIDE.

7. Theory

Port 1 is an 8-bit I/O port. Writing a value to the P1 register controls its output pins.

P1 = 0FEH $\rightarrow$ 11111110B $\rightarrow$ P1.0 = 0 $\rightarrow$ LED OFF

P1 = 0FFH $\rightarrow$ 11111111B $\rightarrow$ P1.0 = 1 $\rightarrow$ LED ON

A software delay based on nested DJNZ loops controls the approximate blinking interval.

8. 8051 Assembly Language Program

```
ORG 0000H
START:
    MOV P1,#0FEH    ; P1.0 = 0, LED OFF
    ACALL DELAY
    MOV P1,#0FFH    ; P1.0 = 1, LED ON
    ACALL DELAY
    SJMP START      ; Repeat continuously
DELAY:
    MOV R7,#0FFH    ; Outer loop
D1:
    MOV R6,#0FFH    ; Inner loop
D2:
    DJNZ R6,D2
    DJNZ R7,D1
    RET
END
```

9. Program Explanation

- ORG 0000H – Sets the program start address at the 8051 reset-vector area.
- MOV P1,#0FEH – Makes P1.0 = 0, turning the LED OFF.
- ACALL DELAY – Calls the delay subroutine.
- MOV P1,#0FFH – Makes P1.0 = 1, turning the LED ON.
- SJMP START – Repeats the blinking sequence continuously.
- DJNZ – Decrements a register and repeats the loop while the result is not zero.

- RET – Returns from the delay subroutine.

10. Steps to Create the Assembly Program and HEX File

1. Open Keil μ Vision or a compatible 8051 Assembly development environment.
2. Create a new project and select AT89C51 or the required compatible 8051 device.
3. Create an Assembly source file with the .asm extension.
4. Enter the program.
5. Save the source file and add it to the project if required.
6. Build/assemble the project.
7. Correct any syntax or configuration errors.
8. Enable HEX file generation in the project output settings if required.
9. Build the project again and locate the generated .hex file.
10. Keep the HEX file ready for loading into SimulIDE.

11. Step-by-Step SimulIDE Interfacing and Simulation Procedure

1. Open SimulIDE and create a new circuit.
2. Place an AT89C51/compatible 8051 microcontroller.
3. Place an LED and approximately 330 Ω resistor.
4. Connect P1.0 (Pin 1) $\rightarrow$ 330 Ω resistor $\rightarrow$ LED anode; connect LED cathode to GND.
5. Connect the 11.0592 MHz crystal between XTAL1 (Pin 19) and XTAL2 (Pin 18).
6. Connect a 33 pF capacitor from each crystal terminal to GND when required.
7. Connect VCC to +5 V and GND to 0 V. Connect EA to +5 V where applicable.
8. Provide the reset arrangement required by the selected model.
9. Verify LED polarity, resistor, power, ground, crystal, EA and reset.
10. Load the generated HEX file into the 8051 component.
11. Set the simulated MCU clock to 11.0592 MHz if available.
12. Run the SimulIDE simulation.
13. Observe the LED turning ON and OFF repeatedly.
14. Change R6/R7 delay values and rebuild/reload the HEX file if required.

Observation Table

Sr. No.	P1.0	LED State	Observation
1	0	OFF	LED OFF during delay
2	1	ON	LED ON during delay
3	0 $\rightarrow$ 1 $\rightarrow$ 0	Blinking	Continuous blinking observed

.Interfacing diagram

8051 ASSEMBLY LANGUAGE PROGRAM FOR BLINKING LED

```
; Program: LED Blinking using 8051
; Crystal Frequency = 11.0592 MHz

ORG 0000H
LJMP START ; Reset vector

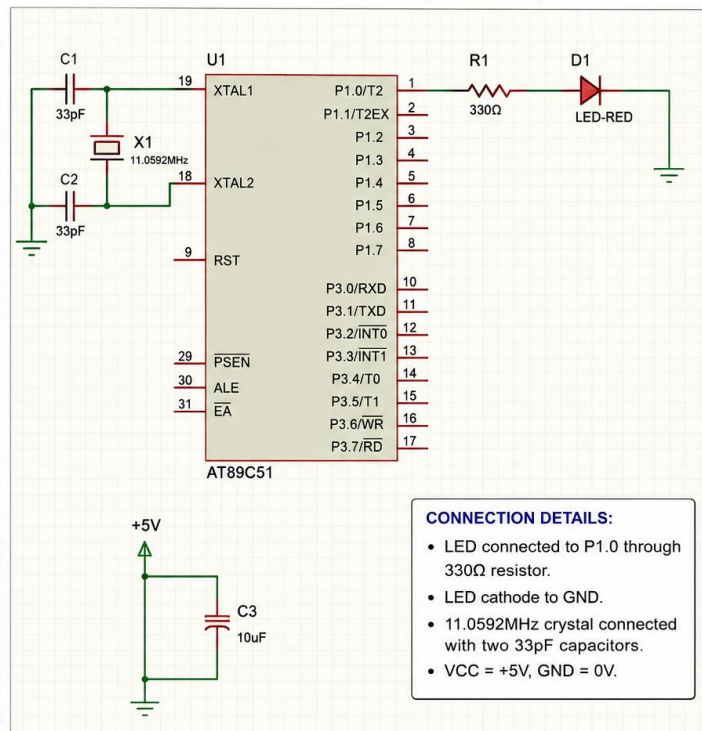
ORG 0030H
START: MOV P1,#0FEH ; P1.0 = 0 (LED ON)
CALL DELAY
MOV P1,#0FFH ; P1.0 = 1 (LED OFF)
CALL DELAY
SJMP START ; Repeat forever

;--- DELAY ROUTINE ---
DELAY: MOV R7,#0FFH ; Outer loop
D1: MOV R6,#0FFH ; Inner loop
D2: DJNZ R6,D2
DJNZ R7,D1
RET
```

PROGRAM EXPLANATION:

- LED is connected to P1.0 pin.
- P1.0 = 0 → LED ON (sinks current)
- P1.0 = 1 → LED OFF
- DELAY routine provides time delay using nested loops.
- LED will blink continuously.

INTERFACING DIAGRAM (SIMULIDE)



Result

The 8051 Assembly Language program for LED blinking was successfully developed. The LED interfacing circuit was created in SimulIDE, the generated HEX file was loaded into the simulated 8051, and continuous LED blinking was observed.

Precautions

- Select the correct 8051 device/model in the assembler and SimulIDE.
- Connect the LED with correct polarity.
- Use a current-limiting resistor such as 330 Ω.
- Use the same crystal frequency in the simulation assumed by the program.
- Ensure the HEX file corresponds to the latest Assembly program.
- Verify VCC, GND, EA, crystal, and reset connections.
- SimulIDE menu names and component properties may vary by version.

Quiz

1. What is the 8051 microcontroller?
2. What is the function of Port 1?
3. Which pin of Port 1 is used for the LED?
4. Why is a resistor connected in series with the LED?

5. What is the purpose of the crystal oscillator?
6. Why is 11.0592 MHz commonly used with the 8051?
7. What is the purpose of ORG 0000H?

Conclusion

Quiz:

1. How to create hex file?
2. How many ports does 8051 have?

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

6. Instruction Proficiency: Evaluate the ability to correctly implement arithmetic and logical instructions in assembly language, focusing on accuracy and precision.
7. Algorithm Efficiency: Assess the efficiency of the algorithms developed, considering the use of arithmetic and logical instructions to manipulate data effectively.
8. Logical Flow: Evaluate the logical flow of the assembly code, ensuring that the sequence of instructions aligns with the desired data manipulation.
9. Error Minimization: Measure the accuracy of the code, emphasizing the reduction of errors in arithmetic and logical operations.
10. Problem Solving: Evaluate problem-solving skills in the context of using arithmetic and logical instructions to address specific computational tasks.

EXPERIMENT No: 9

Arithmetic and Logical Operations Using AVR Microcontroller

Date: _____

Competency and Practical Skills: Ability to develop, assemble, debug, and execute AVR Assembly Language programs for arithmetic and logical operations using an AVR microcontroller and Microchip Studio.

Relevant CO: CO1, CO2

Objectives: To create an AVR Assembly Language project in Microchip Studio, perform arithmetic and logical operations on two 8-bit numbers, debug the program using register inspection, verify the results, and generate the HEX file.

Equipment/Instruments/Software: Microchip Studio for AVR and SAM Devices, AVR Assembler included with Microchip Studio, ATmega328P or compatible AVR microcontroller, compatible programmer/debugger if hardware programming is required, development board/breadboard setup if required, suitable power supply, and Proteus Design Suite as optional simulation software.

1. Aim

To create, write, assemble, debug, and execute an AVR Assembly Language program in Microchip Studio for performing arithmetic and logical operations on two 8-bit numbers.

2. Objectives

- Understand the basic AVR Assembly Language development procedure.
- Create an AVR Assembly project in Microchip Studio.
- Load 8-bit data into AVR registers.
- Perform addition, subtraction, and multiplication.
- Perform bitwise AND, OR, XOR, and NOT operations.
- Observe the results using the register/debug window.
- Generate the HEX file for programming an AVR microcontroller.

3. Introduction to AVR and Related Software

AVR is a family of 8-bit microcontrollers supported by Microchip. AVR microcontrollers provide general-purpose registers, an ALU, timers, ADC, communication interfaces, and programming/debugging facilities. AVR Assembly Language allows the programmer to control the processor at the instruction level.

Microchip Studio is an integrated development environment used to write, assemble, build, debug, and program supported AVR devices. AVR-GCC can be used for C/C++ development, while Proteus may be used for optional circuit simulation. This experiment uses Microchip Studio for AVR Assembly programming.

4. Software and Hardware Requirements

Category	Requirements
Software	Microchip Studio for AVR and SAM Devices; AVR Assembler included with Microchip

	Studio; Proteus Design Suite (optional).
Hardware	AVR microcontroller such as ATmega328P; compatible AVR programmer/debugger if hardware programming is required; development board or breadboard setup if required; suitable power supply.

5. Theory

The AVR architecture contains 32 general-purpose 8-bit registers, R0 to R31. The ALU performs arithmetic and logical operations using these registers. The following instructions are used in this experiment.

Instruction	Function
LDI	Load an immediate value
MOV	Copy data between registers
ADD	Addition
SUB	Subtraction
MUL	Multiplication; result in R1:R0
AND	Bitwise AND
OR	Bitwise OR
EOR	Exclusive OR (XOR)
COM	One's complement (NOT)
RJMP	Relative jump

6. Steps to Write AVR Assembly Code in Microchip Studio

1. Install Microchip Studio for AVR and SAM Devices on the computer.
2. Launch Microchip Studio.
3. Select File → New → Project and select the Assembly/Assembler project option. Enter a suitable project name, for example AVR_Arithmetic_Logical.
4. When the Device Selection window appears, select ATmega328P and click OK.
5. Open the assembly source file with the .asm extension.
6. Type or paste the AVR Assembly Language program given in Section 10.
7. Save the program using File → Save All or Ctrl+S.
8. Build the project using Build → Build Solution or F7.
9. If errors are reported, correct the source code and build again.
10. Start debugging and open the processor/register view.
11. Use Step Into (F11) to execute one instruction at a time and observe register changes.
12. Compare register values with the expected results.
13. After successful build, locate the generated HEX file in the project output/build directory.
14. If hardware is available, use a compatible programmer/debugger to program the generated HEX file into the AVR device.

7. Problem Statement

Write an AVR Assembly Language program for ATmega328P to perform addition, subtraction, multiplication, AND, OR, XOR, and NOT operations on two 8-bit numbers.

A = 12 = 00001100

B = 5 = 00000101

8. Register Assignment

Register	Purpose	Expected Value
R16	Input A	12 (0x0C)
R17	Input B	5 (0x05)
R18	Addition	17 (0x11)
R19	Subtraction	7 (0x07)
R20	AND	4 (0x04)
R21	OR	13 (0x0D)
R22	XOR	9 (0x09)
R23	NOT A	243 (0xF3)
R24	Multiplication LSB	60 (0x3C)
R25	Multiplication MSB	0 (0x00)

9. AVR Assembly Language Program

```

;=====
; Experiment: Arithmetic and Logical Operations
; Microcontroller: ATmega328P
; Language: AVR Assembly
;=====

.include "m328pdef.inc"

; Register Assignment
; R16 = A
; R17 = B
; R18 = Addition result
; R19 = Subtraction result
; R20 = AND result
; R21 = OR result
; R22 = XOR result
; R23 = NOT result
; R24 = Multiplication result (low byte)
; R25 = Multiplication result (high byte)

.cseg
.org 0x0000

RESET:
; Load two 8-bit numbers
; A = 12 = 0x0C
; B = 5 = 0x05
LDI R16, 12
LDI R17, 5

; ADDITION: R18 = A + B
MOVR18, R16
ADD R18, R17

; SUBTRACTION: R19 = A - B
MOVR19, R16
SUB R19, R17

```

```
; MULTIPLICATION: R1:R0 = A * B
MUL R16, R17
MOVR24, R0
MOVR25, R1
```

```
; LOGICAL AND: R20 = A AND B
MOVR20, R16
AND R20, R17
```

```
; LOGICAL OR: R21 = A OR B
MOVR21, R16
OR R21, R17
```

```
; LOGICAL XOR: R22 = A XOR B
MOVR22, R16
EOR R22, R17
```

```
; LOGICAL NOT: R23 = NOT A
MOVR23, R16
COM R23
```

```
LOOP:
    RJMP LOOP
```

10. Program Explanation

10.1 Load Input Values

```
LDI R16, 12
```

```
LDI R17, 5
```

R16 contains A = 12 and R17 contains B = 5.

10.2 Addition

```
MOV R18, R16
```

```
ADD R18, R17
```

$R18 = 12 + 5 = 17$.

10.3 Subtraction

```
MOV R19, R16
```

```
SUB R19, R17
```

$R19 = 12 - 5 = 7$.

10.4 Multiplication

```
MUL R16, R17
```

```
MOV R24, R0
```

```
MOV R25, R1
```

MUL produces a 16-bit result in R1:R0. For $12 \times 5 = 60 = 0x003C$, R25 = 0x00 and R24 = 0x3C.

10.5 AND Operation

00001100

& 00000101

00000100 = 4

The result is stored in R20.

10.6 OR Operation

00001100

| 00000101

00001101 = 13

The result is stored in R21.

10.7 XOR Operation

00001100

^ 00000101

00001001 = 9

AVR uses the EOR instruction for XOR. The result is stored in R22.

10.8 NOT Operation

A = 00001100

~A = 11110011 = 0xF3 = 243

The COM instruction performs one's complement. The result is stored in R23.

11. Expected Results

Operation	Instruction	Decimal Result	Register
Addition	ADD	17	R18
Subtraction	SUB	7	R19
Multiplication	MUL	60	R25:R24
AND	AND	4	R20
OR	OR	13	R21
XOR	EOR	9	R22
NOT A	COM	243	R23

12. Register Values After Execution

Register	Value	Meaning
R16	0C	A = 12
R17	05	B = 5
R18	11	Addition = 17
R19	07	Subtraction = 7
R25	00	Multiplication MSB

R24	3C	Multiplication LSB
R20	04	AND = 4
R21	0D	OR = 13
R22	09	XOR = 9
R23	F3	NOT A = 243

13. Observation Table

Sr. No.	Operation	Expected Result	Observed Result
1	Addition	17	_____
2	Subtraction	7	_____
3	Multiplication	60	_____
4	AND	4	_____
5	OR	13	_____
6	XOR	9	_____
7	NOT A	243	_____

14. Experimental Procedure

1. Create the AVR Assembly project in Microchip Studio using the steps given in Section 6.
2. Select ATmega328P as the target device.
3. Enter and save the Assembly Language program.
4. Build the project using F7.
5. Correct any errors reported by the assembler.
6. Start debugging and open the processor register window.
7. Execute the program step-by-step.
8. Observe R18–R25 and record the values.
9. Compare the observed values with the expected values.
10. Generate the HEX file and, if required, program the physical AVR device.

15. Result

The AVR Assembly Language program was successfully created, assembled, and debugged in Microchip Studio. Addition, subtraction, multiplication, AND, OR, XOR, and NOT operations were performed and the results were verified in the AVR registers.

16. Safety and Necessary Precautions

- Select the correct AVR device before building the project.
- Use the appropriate device include file, such as m328pdef.inc for ATmega328P.
- Check instruction syntax and register names carefully.
- Remember that MUL produces a 16-bit result in R1:R0.
- Save the source program before building and debugging.
- Check values in both hexadecimal and decimal when interpreting register contents.
- Use the correct programmer/debugger and target voltage for hardware programming.

17. Viva Questions

1. What is the purpose of the LDI instruction?
2. Where is the result of the MUL instruction stored?
3. Which AVR instruction performs XOR?
4. What does the COM instruction do?
5. What is a HEX file?

18. Conclusion

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

- Group work while performing experiment on computer.
- Understand the concept while performing coding.
- Understanding of arithmetic and logical operations.
- Debugging and testing skills.
- Quiz answers and submission in time

Experiment No. 10

Read Push Button Switch and Display Its Status on LED

Date _____

Competency and Practical Skills: Ability to understand digital input-output interfacing using the 8051, read a push-button input, control an LED output, and develop and test an 8051 Assembly Language program.

Relevant CO: CO1, CO2

Objectives: To understand digital input and output interfacing with the 8051, read the logic status of a push-button switch, control an LED according to the switch status, and write and execute an 8051 Assembly Language program.

Equipment/Instruments/Software: 8051 microcontroller (AT89C51/compatible), push-button switch, LED, 220 Ω resistor, power supply, connecting wires/breadboard, and 8051 programmer or simulator.

1. AIM

To interface a push-button switch with an 8051 microcontroller, read the status of the switch, and display its status using an LED.

2. OBJECTIVES

- To understand digital input and output interfacing with the 8051.
- To read the logic status of a push-button switch.
- To control an LED according to the switch status.
- To write and execute an 8051 Assembly Language program.

3. APPARATUS / REQUIREMENTS

- 8051 microcontroller (AT89C51/compatible)
- Push-button switch
- LED
- 220 Ω resistor for LED
- Power supply
- Connecting wires / breadboard
- 8051 programmer or simulator

4. THEORY

A push-button switch is a digital input device used to provide a HIGH or LOW logic signal to a microcontroller. In this experiment, the switch is connected to pin P1.0 of the 8051. Port 1 is used as an input port by writing FFH to P1. The LED is connected to P2.0, which is used as an output.

When the push button is not pressed, P1.0 reads logic 1. When the push button is pressed, it is connected to ground and P1.0 reads logic 0. The 8051 checks the input continuously. If P1.0 is LOW, the LED at P2.0 is switched ON; if P1.0 is HIGH, the LED is switched OFF.

5. PIN CONNECTIONS

Device	8051 Pin/Port	Function
Push Button	P1.0	Digital input
LED	P2.0	Digital output
LED resistor	220 Ω in series	Current limiting

6. CIRCUIT DESCRIPTION

Connect one terminal of the push button to P1.0 and the other terminal to GND. Port 1 of the 8051 has internal pull-up capability, so P1.0 remains HIGH when the switch is released. Connect P2.0 to the LED through a 220 Ω current-limiting resistor. The LED indicates the detected switch status.

7. ALGORITHM

1. Start the program.
2. Configure Port 1 as input by loading FFH into P1.
3. Initially turn OFF the LED connected to P2.0.
4. Continuously check the status of P1.0.
5. If P1.0 is HIGH, keep the LED OFF.
6. If P1.0 is LOW, turn the LED ON.
7. Repeat the checking process continuously.

8. 8051 ASSEMBLY LANGUAGE PROGRAM

```
ORG 0000H
MAIN:
    MOV P1, #0FFH    ; Configure Port 1 as input
    CLR P2.0         ; LED initially OFF

CHECK:
    JB P1.0, NOT_PRESSED ; If P1.0 = 1, switch not pressed

    ; Switch is PRESSED
    PRESSED:
        SETB P2.0     ; Turn LED ON
        SJMP CHECK

    ; Switch is NOT PRESSED
    NOT_PRESSED:
        CLR P2.0      ; Turn LED OFF
        SJMP CHECK
END
```

9. PROGRAM EXPLANATION

Instruction	Purpose
MOV P1, #0FFH	Makes Port 1 available for input operation.
CLR P2.0	Turns the LED connected to P2.0 OFF initially.
JB P1.0, NOT_PRESSED	Checks P1.0. If it is 1, control jumps to NOT_PRESSED.
SETB P2.0	Sets P2.0 to 1 and turns the LED ON.
CLR P2.0	Clears P2.0 and turns the LED OFF.
SJMP CHECK	Continuously repeats the switch-status checking.

10. OBSERVATION TABLE

Push Button Status	P1.0	LED at P2.0
Not Pressed	1 (HIGH)	OFF
Pressed	0 (LOW)	ON

RESULT

The push-button switch was successfully interfaced with the 8051 microcontroller. The switch status was read through P1.0 and displayed on the LED connected to P2.0. The LED turns ON when the switch is pressed and turns OFF when the switch is released.

PRECAUTIONS

- Check the power supply and ground connections before running the program.
- Use a suitable resistor in series with the LED to limit current.
- Ensure that the push button is connected correctly to avoid floating input.
- Verify the correct microcontroller device and assembler settings before programming.

CONCLUSION

Quiz:

1. Which port is used to read the push button?
2. Which pin is used to control the LED?
3. Why is FFH written to Port 1?
4. What happens to P1.0 when the switch is pressed?
5. What is the function of the JB instruction?

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

11. Instruction Proficiency: Evaluate the ability to correctly implement arithmetic and logical instructions in assembly language, focusing on accuracy and precision.
12. Algorithm Efficiency: Assess the efficiency of the algorithms developed, considering the use of arithmetic and logical instructions to manipulate data effectively.
13. Logical Flow: Evaluate the logical flow of the assembly code, ensuring that the sequence of instructions aligns with the desired data manipulation.
14. Error Minimization: Measure the accuracy of the code, emphasizing the reduction of errors in

arithmetic and logical operations.

15. Problem Solving: Evaluate problem-solving skills in the context of using arithmetic and logical instructions to address specific computational tasks.

Experiment no: 11

Interfacing Buzzer with AVR Board

Date: _____

Competency and Practical Skills: Competency in AVR microcontroller programming and practical skill in interfacing an output device (buzzer) with a digital I/O port will be achieved.

Relevant CO: CO2

Objectives: To interface a buzzer with an AVR ATmega32 microcontroller and develop a C program to control the buzzer in ON/OFF mode at regular intervals.

Equipment/Instruments/Software:

- AVR ATmega32 development board / ATmega32 microcontroller
- Active buzzer
- 220 Ω resistor
- Connecting wires
- 5 V DC supply
- Microchip Studio / AVR-GCC
- SimulIDE software

Theory:

A buzzer is an electro-acoustic output device used to generate an audible sound. In an AVR-based system, a buzzer can be controlled through a digital output pin of the microcontroller. When the selected port pin is driven HIGH, the buzzer is activated; when the pin is driven LOW, the buzzer is deactivated.

For this experiment, PB0 of the ATmega32 is configured as an output using the DDRB register. The PORTB register is then used to set or clear PB0. A 220 Ω resistor is included in the interface as a current-limiting/protection element. An active buzzer is used so that applying the required DC level produces the audible tone.

The program uses a delay of 1 second between the ON and OFF states. Therefore, the buzzer repeatedly turns ON for one second and OFF for one second.

Circuit Diagram / Interfacing Diagram (SimulIDE):

EXPERIMENT: INTERFACING BUZZER WITH AVR BOARD

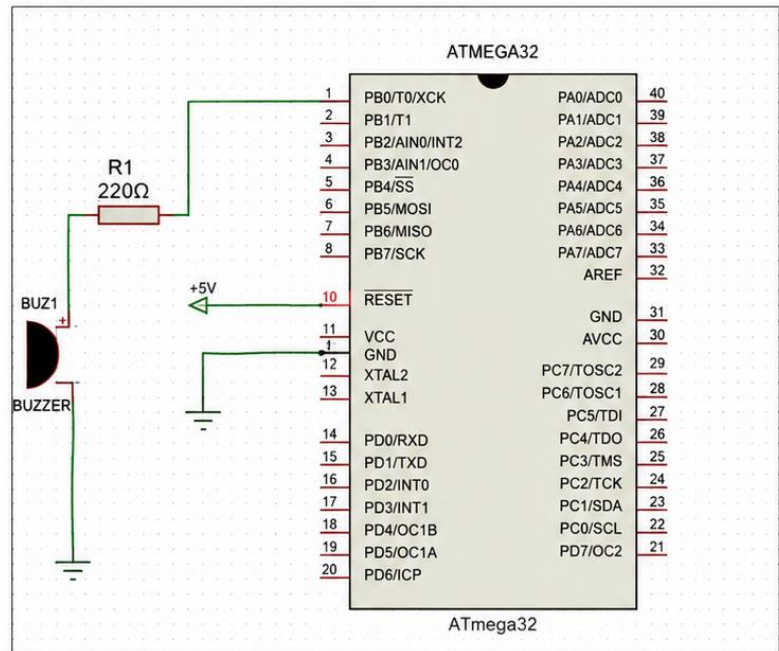
1. AIM

To interface a buzzer with AVR ATmega32 microcontroller and write a program to turn the buzzer ON and OFF at regular intervals.

2. COMPONENTS REQUIRED

Sr. No.	Component	Quantity
1	AVR ATmega32 Development Board	1
2	Buzzer (Active)	1
3	Resistor 220 Ω	1
4	Connecting Wires	As required
5	5V DC Supply	1
6	SimulIDE Software	1

3. CIRCUIT DIAGRAM (SimulIDE)



4. PROGRAM IN C (AVR-GCC)

```
1 #include <avr/io.h>
2 #include <util/delay.h>
3 int main(void)
4 {
5     DDRB |= (1<<PB0); // PB0 as output
6     while(1)
7     {
8         PORTB |= (1<<PB0); // Buzzer ON
9         _delay_ms(1000); // ON for 1 second
10        PORTB &= ~(1<<PB0); // Buzzer OFF
11        _delay_ms(1000); // OFF for 1 second
12    }
13 }
```

6. PROCEDURE

1. Open SimulIDE software.
2. Create a new circuit and place ATmega32, Buzzer, Resistor (220 Ω) and 5V, GND.
3. Make the connections as shown in the circuit diagram.
4. Write the above program in AVR-GCC and compile it to generate .hex file.
5. Load the .hex file into ATmega32 (Double click on MCU → Program → Load File).
6. Run the simulation.
7. Observe the buzzer ON and OFF at 1 second interval.

9. VIVA QUESTIONS

1. What is the purpose of a buzzer?
2. Why is a resistor used with the buzzer?
3. Which register is used to set the direction of an AVR port?
4. What is the difference between active and passive buzzer?
5. How can we generate different tones with a buzzer?

5. PROGRAM EXPLANATION

- `DDRB |= (1<<PB0);` ----- Configures PB0 as output.
- `PORTB |= (1<<PB0);` ----- Makes PB0 HIGH, buzzer ON.
- `_delay_ms(1000);` ----- Buzzer ON for 1 second.
- `PORTB &= ~(1<<PB0);` ----- Makes PB0 LOW, buzzer OFF.
- The process repeats in an infinite loop.

7. OBSERVATION

The buzzer turns ON for 1 second and OFF for 1 second repeatedly.

8. RESULT

Thus, the buzzer is successfully interfaced with AVR ATmega32 board and its ON/OFF operation is verified.

Pin Connections:

Sr. No.	AVR ATmega32 / Component	Connection
1	PB0 (Pin 1)	Connected to buzzer (+) through 220 Ω resistor
2	Buzzer (-)	Connected to GND
3	VCC (Pin 11)	+5 V supply
4	RESET (Pin 9)	Pulled HIGH to +5 V
5	GND (Pin 31)	Common ground

Program in C (AVR-GCC):

```
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRB |= (1 << PB0);    // PB0 as output

    while (1)
    {
        PORTB |= (1 << PB0); // Buzzer ON
        _delay_ms(1000);

        PORTB &= ~(1 << PB0); // Buzzer OFF
        _delay_ms(1000);
    }

    return 0;
}
```

Program Explanation:

Statement	Function
DDRB = (1 << PB0);	Configures PB0 as an output pin.
PORTB = (1 << PB0);	Sets PB0 HIGH and activates the buzzer.
_delay_ms(1000);	Keeps the buzzer in its current state for 1 second.
PORTB &= ~(1 << PB0);	Clears PB0 and deactivates the buzzer.
while(1)	Repeats the ON/OFF sequence continuously.

Procedure:

- Open Microchip Studio and create an AVR C project for ATmega32, or prepare the program using an AVR-GCC toolchain.
- Enter the above C program and compile/build the project to generate the HEX file.
- Open SimulIDE and create a new circuit.
- Place ATmega32, active buzzer, 220 Ω resistor, +5 V and GND components.
- Make the connections according to the interfacing diagram.
- Load the generated HEX file into the ATmega32 microcontroller in the simulator.
- Run the simulation and observe the buzzer.
- Verify that the buzzer remains ON for approximately 1 second and OFF for approximately 1 second repeatedly.

Observation:

Sr. No.	PB0 State	Buzzer State	Observation
1	HIGH	ON	Buzzer produces audible sound
2	LOW	OFF	Buzzer is silent
3	Repeating	ON/OFF	ON/OFF operation repeats every 1 second

Result:

The buzzer was successfully interfaced with the AVR ATmega32 board. The program successfully controlled the buzzer by making PB0 HIGH and LOW at one-second intervals, and the operation was verified in the simulation.

Safety and Necessary Precautions:

- Check the polarity of the buzzer before powering the circuit.
- Ensure that the AVR board and simulator circuit have a common ground.
- Do not apply a voltage higher than the specified supply voltage to the ATmega32.
- Use the resistor in the interface as shown in the circuit diagram.
- Do not make or change connections while the powered hardware circuit is operating.
- Verify the generated HEX file and selected microcontroller before running the simulation.

Conclusion:

The experiment demonstrates digital output control using the AVR ATmega32. By configuring PB0 as an output and controlling the PORTB register, an active buzzer can be switched ON and OFF. The same method can be extended to other digital output devices.

Quiz / Viva Questions:

1. What is the function of a buzzer in an embedded system?
2. Why is PB0 configured as an output in this experiment?
3. What is the purpose of the DDRB register?
4. What is the difference between an active buzzer and a passive buzzer?
5. What is the function of PORTB in this program?

Rubrics	1	2	3	4	5
Marks					

Rubrics:

1. Execution Efficiency: Assess the effectiveness of the program in controlling the buzzer with the required timing.
2. Logical Flow and Control: Evaluate the correctness and clarity of the ON/OFF control sequence.
3. Interfacing Accuracy: Evaluate the correctness of AVR, resistor, buzzer, supply and ground connections.
4. Code Implementation: Assess proper use of DDRB, PORTB and delay instructions.
5. Overall Proficiency: Determine the student's overall competence in AVR programming and hardware interfacing.

EXPERIMENT No: 12

Interfacing 7-Segment LED Display with AVR ATmega32 using SimulIDE

Date:

Competency and Practical Skills: Ability to configure AVR GPIO ports, interface LED-based display devices, develop embedded C programs, and use simulation tools for microcontroller-based systems.

Relevant CO: CO1, CO2

Objectives: To configure PORTD of ATmega32 as an output port, interface a common-cathode 7-segment display, generate segment codes for digits 0–9, and simulate the display operation in SimulIDE.

Equipment/Instruments/Software: ATmega32 AVR microcontroller, common-cathode 7-segment LED display, 330 Ω current-limiting resistors, 8 MHz crystal, 22 pF capacitors, 10 k Ω resistor and reset switch, 5 V DC supply/GND, and SimulIDE.

1. Aim

To interface a common-cathode 7-segment LED display with an AVR ATmega32 microcontroller in SimulIDE and display the digits 0 to 9 sequentially.

2. Objectives

- Configure PORTD of ATmega32 as an output port.
- Interface the seven segments and decimal-point LED through current-limiting resistors.
- Generate the required segment codes for digits 0–9.
- Simulate the circuit in SimulIDE and observe the displayed digits.

3. Apparatus / Software Required

Sr. No.	Component / Software	Quantity
1	ATmega32 AVR microcontroller	1
2	Common-cathode 7-segment LED display	1
3	330 Ω current-limiting resistor	8
4	8 MHz crystal	1
5	22 pF capacitor	2
6	10 k Ω resistor and reset switch	1 each
7	5 V DC supply / GND	As required
8	SimulIDE	1

4. Theory

A 7-segment display contains seven individual LED segments named a, b, c, d, e, f and g. An optional decimal-point segment is called dp. In a common-cathode display, the cathode terminals are connected together and connected to ground. A segment glows when its anode is driven HIGH.

Therefore, the AVR can control each segment from an output port, with a current-limiting resistor in series with every segment.

In this experiment, ATmega32 PORTD is used. PD0–PD7 are mapped to a–g and dp respectively. The program writes a predefined hexadecimal code to PORTD for each digit and waits one second before displaying the next digit.

5. SimulIDE Schematic Connection

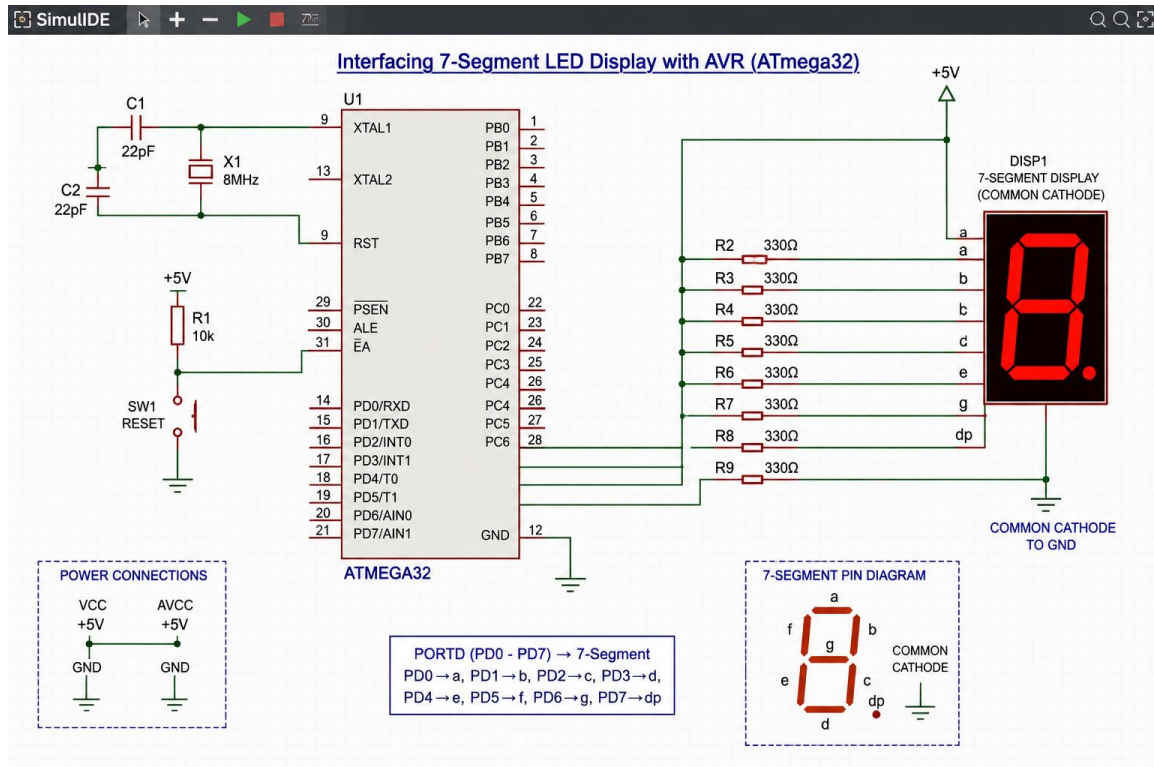


Figure 1: Original SimulIDE-style connection of ATmega32 with a common-cathode 7-segment display.

6. Pin Connection Table

7-Segment Segment	ATmega32 Port	Connection
A	PD0	Through 330 Ω resistor
B	PD1	Through 330 Ω resistor
C	PD2	Through 330 Ω resistor
D	PD3	Through 330 Ω resistor
E	PD4	Through 330 Ω resistor
F	PD5	Through 330 Ω resistor
G	PD6	Through 330 Ω resistor
Dp	PD7	Through 330 Ω resistor
Common Cathode	GND	Ground

7. AVR C Program

```
#define F_CPU 8000000UL

#include <avr/io.h>
#include <util/delay.h>

// Common-cathode 7-segment codes
```

```

// Bit pattern: dp g f e d c b a

unsigned char seg_code[10] =
{
    0x3F, // 0
    0x06, // 1
    0x5B, // 2
    0x4F, // 3
    0x66, // 4
    0x6D, // 5
    0x7D, // 6
    0x07, // 7
    0x7F, // 8
    0x6F  // 9
};

int main(void)
{
    unsigned char i;

    // Configure PORTD as output
    DDRD = 0xFF;

    while (1)
    {
        // Display digits 0 to 9
        for (i = 0; i < 10; i++)
        {
            PORTD = seg_code[i];
            _delay_ms(1000);
        }
    }

    return 0;
}

```

8. Procedure

1. Open SimulIDE and create a new circuit.
2. Place an ATmega32 microcontroller and set its clock frequency to 8 MHz.
3. Place a common-cathode 7-segment display.
4. Connect PD0–PD7 to segments a–g and dp through 330 Ω resistors.
5. Connect the common cathode of the display to GND.
6. Connect the ATmega32 power pins to +5 V and GND as required by the SimulIDE model.
7. Add the 8 MHz crystal and capacitors if using the external-clock arrangement shown in the schematic.
8. Compile the AVR C program and load the generated HEX file into the ATmega32 in SimulIDE.
9. Run the simulation and observe the display.
10. Verify that the display cycles through 0, 1, 2, ..., 9 and repeats.

9. Expected Observation

Sequence	Expected Display
1	0
2	1
3	2
4	3
5	4
6	5
7	6
8	7
9	8
10	9
Repeat	0 → 9 continuously

10. Result

The common-cathode 7-segment LED display was successfully interfaced with the AVR ATmega32 in SimulIDE. The program displayed the digits 0 to 9 sequentially with a delay of one second between consecutive digits.

11. Precautions

- Use a current-limiting resistor for each LED segment.
- Confirm that the display is common-cathode; common-anode displays require inverted logic.
- Ensure the ATmega32 clock setting matches F_CPU in the program.
- Check all PORTD-to-segment connections before starting the simulation.
- Do not short LED segments directly to the supply.

12. Quiz:

1. What is a 7-segment display?
2. What is the difference between common-cathode and common-anode displays?
3. Why are 330 Ω resistors used with the LED segments?
4. Which ATmega32 port is used in this experiment?
5. What does `DDRD = 0xFF` do?
6. Why is `F_CPU` defined as 8 MHz?
7. What hexadecimal code is used to display digit 8?

13. Conclusion

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks						

Rubrics:

- 1) Instruction Proficiency: Evaluate the ability to correctly implement arithmetic and logical instructions in assembly language, focusing on accuracy and precision.
- 2) Algorithm Efficiency: Assess the efficiency of the algorithms developed, considering the use of arithmetic and logical instructions to manipulate data effectively.
- 3) Logical Flow: Evaluate the logical flow of the assembly code, ensuring that the sequence of instructions aligns with the desired data manipulation.
- 4) Error Minimization: Measure the accuracy of the code, emphasizing the reduction of errors in arithmetic and logical operations.
- 5) Problem Solving: Evaluate problem-solving skills in the context of using arithmetic and logical instructions to address specific computational tasks.